

1. Qu'est-ce qu'une Image ? (Origines et Perception)

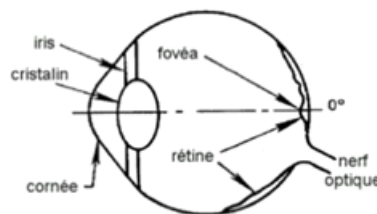
1.1. Étymologie et Évolution Historique

- Le terme vient du latin *imaginem* (accusatif de *imago*), qui signifie représentation, portrait, apparence, ou fantôme.
- En français, le mot a d'abord pris le sens de statue.
- Au 12e siècle, le terme évolue pour désigner la représentation graphique d'un objet ou d'une personne.
- En 1690, le dictionnaire historique de la langue française (sous la direction d'Alain Rey, éditions Le Robert) y associe la notion de représentation immobile, donnant naissance à l'expression « Sage comme une image ».
- Aujourd'hui, selon le Wiktionnaire, une image est la représentation d'êtres ou d'objets par le biais du dessin, de la peinture, de la sculpture, de la gravure, de la photographie, du cinéma, etc..

1.2. Le Concept de Perception

- Dans le cadre de ce cours, le but est d'essayer de représenter ce que l'on voit et ce que l'on perçoit.
- La perception est définie (selon Wikipedia) comme l'activité par laquelle un sujet (nous-mêmes) fait l'expérience d'objets ou de propriétés qui sont présents dans son environnement.
- La lumière est réfléctie par les surfaces avec une certaine longueur d'onde. Les longueurs d'onde visibles se situent entre 400nm et 700nm sur le spectre électromagnétique.

1.3. Le Système Visuel Humain



- L'œil agit comme un système qui perçoit la lumière pour former une image qui est ensuite transmise au cerveau. **Cristallin** : C'est une lentille dont le rôle est de diriger les rayons lumineux directement sur la rétine, qui est la couche photosensible de l'œil.
- **Iris** : Il contrôle le diamètre de la pupille, ce qui permet de réguler la quantité d'énergie lumineuse qui arrive sur la rétine.
- **La rétine** (qui contient également la fovéa, positionnée à 0°) est reliée au nerf optique. Elle est composée de deux types de photorécepteurs :
 - **Les cônes** : Ils permettent principalement la vision chromatique (la couleur).
 - **Les bâtonnets** : Ils assurent une bonne réponse temporelle et possèdent une grande sensibilité (notamment pour la perception des niveaux de gris).

3. Le Modèle Mathématique de l'Image Numérique

Pour pouvoir traiter une image informatiquement, il faut la traduire en langage mathématique. On représente ainsi une image par une fonction mathématique :

$$f : \Omega \rightarrow X^c$$

3.1. L'Espace d'Entrée (Ω)

- Ω = l'espace d'étude (le domaine)
- x = la variable d'espace
- $x \in \Omega$
- Cet espace peut prendre plusieurs dimensions, étant souvent **bi-dimensionnel** (pour une

image 2D) ou tri-dimensionnel (pour une image 3D).

- La nature de cet espace peut être :
 - **Continu** : $\Omega = \mathbb{R}^n$
 - **Discret** : $\Omega = \mathbb{Z}^n$ ou plus restrictivement $\Omega = \mathbb{N}^n$
- Dans ce cours, l'attention est portée principalement sur les **images discrètes 2D utilisant des index positifs**, modélisées par :
 $I : \mathbb{N}^2 \rightarrow \mathbb{N}^c$
- Par convention :
 - On utilise les **index (i, j)** pour les images discrètes
 - Les **coordonnées (x, y)** sont réservées aux images continues.

3.2. L'Espace de Sortie ($\mathbb{X}^c$)

- $\mathbb{X}^c$ c'est l'espace de sortie qui permet de représenter visuellement l'image.
- La valeur **X** représente le type de données, qui est généralement :
 - **Continu** : $X = \mathbb{R}$
 - **Discret** : $X = \mathbb{Z}$ ou $X = \mathbb{N}$

4. Les Étapes de la Numérisation d'une Image

La numérisation passe par **deux étapes fondamentales** :

1. **L'échantillonnage** (qui concerne l'espace spatial)
2. **La quantification** (qui concerne les valeurs de couleur ou de luminosité)

4.1. Échantillonnage (Discrétisation de l'Espace)

- Si $\Omega = \{\mathbb{R}\}^n \rightarrow$ **continu**
☞ il faut **échantillonner**
- Si $\Omega = \{\mathbb{Z}\}^n$ ou $\{\mathbb{N}\}^n \rightarrow$ **déjà discret**
☞ **pas d'échantillonnage nécessaire**
- L'espace réel étant **continu**, il faut effectuer un **pavage** de cet espace pour obtenir une représentation **discrète**. Cette opération s'appelle **l'échantillonnage**.
- Un **pavage** est une partition de l'espace continu en différentes **cellules élémentaires**.
- Une cellule élémentaire est appelée :
 - **Pixel en 2D**
 - **Voxel en 3D**
- Ces pavages sont généralement **réguliers**, ce qui signifie que toutes les cellules ont **la même taille et les mêmes angles**.

Selon **I. Bloch**, il existe **trois pavages réguliers du plan** :

- pavage triangulaire
- pavage carré
- pavage hexagonal

Dans la pratique, **les pavages utilisés sont presque toujours carrés**.

- L'espace d'une image peut être :
 - **non borné** (image à support infini)
 - **borné** (image à support fini)

Le traitement des images discrètes s'intéresse surtout aux **images à support fini**.

- Les **dimensions d'une image** sont définies par :
 - la **hauteur**
 - la **largeur**
 - la **profondeur** (dans le cas d'une image 3D)

on doit toujours faire l'échantillonnage ?

- Si l'image vient du monde réel $\rightarrow$ **oui**
- Si l'image est déjà numérique $\rightarrow$ **non (c'est déjà fait)**

En pratique, **On travaille presque toujours avec des images discrètes**.

4.2. Les Canaux de Couleur (c)

- La variable **c** correspond au **nombre de canaux de l'image**.
- Si **c = 1**, l'image est en **niveaux de gris** (espace mono-dimensionnel perçu par les bâtonnets).
- Si **c = 3**, l'image est en **couleur**.

Depuis le **19^e siècle**, la théorie de **Young-Helmholtz** explique que l'œil humain contient **trois types de cônes sensibles à différentes longueurs d'onde lumineuses** :

- **Cônes longs (L)** : sensibles au jaune-vert (~585 nm)
- **Cônes moyens (M)** : sensibles autour du vert (~550 nm)
- **Cônes courts (S)** : sensibles autour du bleu (~430 nm)

Cela donne naissance à l'**espace de couleur RGB** :

- **R** : Rouge
- **G** : Vert (Green)
- **B** : Bleu

4.3. Quantification (Discrétisation de la Représentation)

La quantification consiste à **discrétiser l'espace de représentation**, qu'il soit dans **N**, **Z** ou **R**.

Les quantifications les plus courantes sont :

- **Dans N** :
intervalle **[0 , 255]**
→ ce qui donne **256 valeurs possibles**
(quantification non régulière)
- **Dans R** :
intervalle **[0 , 1]**
encodé en **float16, float32 ou float64**
(quantification non régulière)

Quantification régulière :

- pas constant entre niveaux
- ex : entiers 0 → 255

Quantification non régulière :

- pas variable
- ex : floats (float16, float32, float64)

Conventions :

- **0** représente le **noir**
- La valeur maximale (**1 ou 255**) représente :
 - le **blanc**
 - ou une **couleur pure (rouge, vert, bleu)** dans le modèle RGB

5. Synthèse et Représentation Matricielle

5.1. Les 4 Types d'Images Numériques

Le cours se concentre sur **quatre combinaisons d'espaces de représentation** :

1. **$I : \mathbb{N}^2 \rightarrow \mathbb{Z}^1$**
Image discrète bi-dimensionnelle à **valeurs entières en niveaux de gris**
2. **$I : \mathbb{N}^2 \rightarrow \mathbb{Z}^3$**
Image discrète bi-dimensionnelle à **valeurs entières en couleur**
3. **$I : \mathbb{N}^2 \rightarrow \mathbb{R}^1$**
Image discrète bi-dimensionnelle à **valeurs continues en niveaux de gris**
4. **$I : \mathbb{N}^2 \rightarrow \mathbb{R}^3$**
Image discrète bi-dimensionnelle à **valeurs continues en couleur**

5.2. La Matrice de l'Image

Dans le cas le plus simple d'une **image en niveaux de gris** :

$$I : \mathbb{N}^2 \rightarrow \mathbb{Z}$$

avec :

- **H** = hauteur
- **W** = largeur

On a :

Pour tout i dans $[0, H]$

Pour tout j dans $[0, W]$

$I(i, j)$ appartient à Z

Cette structure mathématique **forme une matrice**.

Exemple

Le cours illustre ce concept avec une matrice représentant un **visage souriant (smiley)** :

- les pixels du dessin prennent la valeur **255 (blanc)**
- le fond prend la valeur **0 (noir)**

Les axes i et j représentent les **coordonnées des pixels**.

Dans cet exemple :

- l'échantillonnage utilise un **pavage carré**
- la taille de l'image est **32 × 32 pixels**
- la quantification utilise **256 valeurs entières**

Semaine 2

Friday, 6 March 2026 15:08

1. Les objectifs du traitement des images

Le traitement d'images a trois grands objectifs :

1. Analyser les images
2. Transformer les images numériques
3. Produire des images

Objectif 1 : Analyser des images

Reconnaissance de formes (Pattern Recognition)

La reconnaissance de formes consiste à **découvrir automatiquement des régularités dans les données** à l'aide d'algorithmes afin de :

- classer les données
- prendre des décisions.

Définition selon **Bishop**.

Exemple

Détection de l'eau à partir des données du **satellite SWOT**.

Les algorithmes peuvent être :

- **supervisés** (apprentissage à partir de données annotées)
- **non supervisés**

Types de tâches

1. Classification

Attribuer une **étiquette globale** à l'image.

Exemple :

Un réseau de neurones **CNN** identifie l'image comme "**chat**".

2. Détection d'objets

Localiser un objet dans l'image, souvent à l'aide de **boîtes englobantes (bounding boxes)**.

3. Régression

Prédire une **valeur numérique continue**.

Exemple :

Compter le **nombre de chats** dans l'image.

4. Segmentation sémantique

Classifier **chaque pixel** de l'image.

Extraction d'information

Transformer les **données brutes** en **informations de haut niveau**.

Cela peut produire :

- des **informations quantitatives**
(ex : volume d'eau)
- une **préparation à l'action**
(ex : systèmes pour voitures autonomes)
- une **aide à la décision**
(ex : détection d'une pathologie médicale)

Objectif 2 : Transformer les images numériques

Photographie computationnelle

La photographie computationnelle regroupe les techniques de **capture et de traitement d'images utilisant le calcul numérique** au lieu de simples processus optiques.

Cela permet :

- d'améliorer les capacités des appareils
- de réduire les coûts ou la taille des capteurs
- d'ajouter de nouvelles fonctionnalités impossibles avec la photographie argentique.

Exemples d'acquisition

- Imagerie à grande gamme dynamique **HDR**
- **Panoramas**
- **Champ lumineux (Light Field)**
- **Capteurs LiDAR**
(par exemple utilisés pour cartographier les catacombes de Paris)

Améliorer la qualité de l'image

Plusieurs techniques permettent d'améliorer la qualité :

- **Débruitage de l'image** (Talbot et al., 2009)
- **Amélioration et manipulation des couleurs**
Exemple : passage de l'espace RGB vers YUV ou **niveaux de gris**
- **Suppression du flou** (Deblurring)

Modifier l'aspect visuel

Certaines techniques servent à modifier l'apparence d'une image :

- Application de **filtres**
(exemple : filtres humoristiques sur les visages)
- **Inpainting**
Reconstruction ou modification d'une partie de l'image.

Exemples :

- Effacer un **logo sur un avion Aeroflot**
- Cas historique de **censure** : effacement de **Nikolai Yezhov** à côté de **Joseph Staline**

Encoder les images

L'objectif est de **stocker l'information efficacement** grâce à la compression (avec ou sans perte).

Une image couleur non compressée en **uint8** occupe beaucoup d'espace :

- 1 pixel = **3 valeurs** (3 canaux RGB)
- donc **3 octets par pixel**

Un capteur de **48 mégapixels** produit une image d'environ **144 Mo**.

Cela signifie que **1 Go permet de stocker environ 7 photos**.

Les algorithmes de compression encodent généralement :

- **les informations globales de l'image** (basses fréquences)
- **les variations locales** (hautes fréquences)

Objectif 3 : Produire des images

À partir de modélisations (Rendering)

Exemple : **Ray-tracing** (lancer de rayons).

Cette technique simule mathématiquement le **parcours de la lumière** :

- View Ray
- Shadow Ray

depuis la source lumineuse jusqu'à la caméra.

À partir d'instructions

Utilisation de **l'intelligence artificielle générative**.

Exemple :

DALL-E 3

Prompt possible :

"Des étudiants qui écoutent un professeur dans un amphithéâtre".

2. Chaîne de traitement

Les processus appliqués à une image **ne sont pas indépendants**.

Ils s'enchaînent pour former un **système complet**, allant :

de l'acquisition des données

jusqu'à la prise de décision finale.

Structure d'une chaîne de traitement

Selon le **niveau d'information manipulé** :

Formation de l'image

Niveau Pixel :

- **Numérisation** : Conversion de l'image réelle en valeurs numériques (pixels avec intensité 0–255).
- **Pré-traitements** : Amélioration de l'image : réduction du bruit, amélioration du contraste, égalisation d'histogramme, etc.
- **Segmentation** : Séparation de l'image en parties simples (ex : séparer objet et fond).

Niveau Régions

- **Segmentation**
- **Post-traitements** : Nettoyer les régions obtenues (supprimer petites erreurs, lisser les contours).
- **Extraction de primitives** : Extraire des caractéristiques simples d'une région : contours, aire, forme, couleur.

Niveau Image (global)

- **Extraction de primitives**
- **Extraction d'information** : Interpréter les caractéristiques pour comprendre l'image (ex : reconnaître un objet, détecter un visage).

3. Évaluation des systèmes

L'objectif est de **prédire les performances d'un système sur de nouvelles données**.

Exemple :

annoncer une **marge d'erreur de X %** pour différencier les chats et les chiens.

Protocoles d'évaluation

Cas non supervisé

L'algorithme est testé directement sur une **base de données de test**.

Cette base doit être **représentative du monde réel**.

Problème

Les bases de données contiennent souvent des **biais** :

- plus de photos de chats
- chiens photographiés sur fond clair

Les algorithmes nécessitent aussi **un réglage de paramètres**.

Solution :

Base de validation

Cas supervisé

Les données sont divisées en **trois ensembles distincts** :

Base d'apprentissage

Utilisée pour **entraîner l'algorithme**.

Base de validation

Utilisée pour **choisir les paramètres optimaux**.

Base de test

Utilisée uniquement pour **évaluer les performances finales**.

Règle d'or

Les résultats sur la **base de test** doivent être calculés **une seule fois**.

Métriques d'évaluation

Les performances sont mesurées en comparant les prédictions de l'algorithme à une **vérité terrain (ground truth)**.

Cette vérité est généralement **annotée manuellement**.

Classification binaire (ex : Eau / Non-eau)

On utilise une **matrice de confusion** :

- **TP** : Vrais positifs
- **FP** : Faux positifs
- **FN** : Faux négatifs
- **TN** : Vrais négatifs

		Actual class	
		P	N
Predicted class	P	TP	FP
	N	FN	TN

Formules

Précision :

$$P = TP / (TP + FP) \text{ (Positives)}$$

Rappel :

$$R = TP / (TP + FN) \text{ (True Positives)}$$

Taux d'erreur :

$$E = (FP + FN) / (TP + FP + FN + TN) \text{ (All)}$$

Score F1 :

$$F1 = 2 \times (P \times R) / (P + R)$$

Régression (prédiction d'une valeur)

Soit :

- y_i = prédiction de l'algorithme
- $\hat{y}_i$ = valeur réelle
- N = nombre d'images

MAE (Mean Absolute Error)

MAE = somme de $|y_i - \hat{y}_i|$

MSE (Mean Squared Error)

MSE = somme de $(y_i - \hat{y}_i)^2$

Exemple comparatif

Vérités terrain :

{2, 5, 4, 7, 3}

Système 1

Prédictions :

{2, 4, 4, 12, 3}

MAE = 6 / 5

MSE = 26 / 5

Système 2

Prédictions :

{3, 4, 5, 5, 4}

MAE = 6 / 5

MSE = 8 / 5

Conclusion :

Le **Système 2** est meilleur, car il est **moins sensible aux grosses erreurs**.

Introduction au Traitement des Images et à son Évaluation

Un système de traitement des images vise à **extraire de l'information** pour diverses applications courantes :

- Appliquer des filtres sur un smartphone.
- Détecter quels sont les objets présents dans une image.

Lorsqu'on développe un tel système, il est **extrêmement important de l'évaluer**. En effet, à l'exception des tâches très triviales, un algorithme ne fonctionnera jamais à 100 % et produira des erreurs. Le but de l'évaluation est d'être capable d'**annoncer des performances fiables** (une marge d'erreur précise) afin que les utilisateurs ou les clients sachent à quoi s'attendre.

1. La Tâche de Classification : Exemple du Chien et du Chat

Pour illustrer le propos, le cours prend l'exemple d'un algorithme visant à différencier les chats des chiens.

- **Utilité pratique** : Bien que cela paraisse simple, c'est ce qui est utilisé dans les smartphones pour classer automatiquement les photos (ex: si vous tapez "chat" dans votre galerie, l'application vous ressort toutes les photos contenant un chat).
- **Définition de la classification** : C'est une tâche qui consiste à **assigner une classe** à une image dans son ensemble pour décrire ce qu'elle contient.
- **Le besoin de précision** : Annoncer "mon système a un taux d'erreur de 10 %" n'est pas suffisant. Il faut être plus précis : est-ce que 10 % des chats sont pris pour des chiens ? 10 % des chiens pour des chats ? Ou 10 % d'erreurs au global ? (Le calcul précis de ces métriques fera l'objet d'un autre cours abordant la classification et la régression).

2. Les Algorithmes Non Supervisés et le Problème des Biais

Il est essentiel de revoir les fondamentaux de l'évaluation, car ils sont souvent mal respectés, même par les professionnels du domaine. Il existe deux grands types d'algorithmes : supervisés et non supervisés.

A. Qu'est-ce qu'un algorithme non supervisé ?

- C'est un algorithme qui **n'apprend pas à partir de données**.
- Il repose sur des **a priori** (des règles ou opérations décidées à l'avance par le concepteur).
- **Exemple fictif** : Faire un *clustering* (regroupement) sur l'image, puis appliquer un seuil sur la taille (l'aire) des clusters restants pour décider si l'animal est un chat ou un chien.
- C'est un algorithme qui **apprend à partir de données non étiquetées** (sans réponses connues).
- Il cherche à **découvrir des structures ou des régularités** dans les données (regroupements, similarités, anomalies...).

B. La Base de Test et la Notion de Biais

Même sans apprentissage, il faut évaluer cet algorithme. Pour cela, on utilise une **Base d'images de test** (où l'on connaît l'image en entrée et le résultat attendu).

- **Idéalement** : Cette base devrait être parfaitement **représentative du monde réel** (toutes les races, toutes les couleurs, tous les fonds possibles).
- **Dans la réalité** : C'est impossible d'avoir toutes les images existantes. On utilise donc un échantillon.
- **Les biais** : Lorsqu'un échantillon n'est pas totalement représentatif, il introduit des biais. Par exemple, dans la base montrée dans la vidéo :
 - Il y a plus de chats que de chiens.
 - Les chats sont sur des fonds sombres et les chiens sur des fonds clairs.

C. Le Danger de l'Évaluation Directe sur la Base de Test

Même un algorithme non supervisé possède des **paramètres à régler** (ex: la valeur du seuil pour le clustering, ou le seuil de l'aire).

- **Le problème** : Si vous réglez ces paramètres directement sur la base de test, vous allez **augmenter artificiellement vos performances** en adaptant l'algorithme aux biais spécifiques de cette base (ex: l'algorithme déduirait qu'un fond sombre équivaut à un chat).
- **La conséquence** : Lorsque vous vendrez l'algorithme au client (qui annonçait 95 % de réussite), il obtiendra des performances catastrophiques sur de nouvelles images car le système ne saura pas gérer des situations non biaisées de la même manière. Le client sera mécontent.

3. La Solution : La Base d'Images de Validation

Pour remédier au problème d'adaptation aux biais de la base de test, on introduit une deuxième base de données.

- **La Base de Validation** : C'est sur cette base que l'on va **régler et chercher les bons paramètres** de notre algorithme pour en maximiser les performances.
- **Le rôle de la Base de Test** : Une fois que l'on est satisfait des réglages effectués sur la base de validation, on teste l'algorithme sur la base de test pour évaluer sa capacité à **généraliser** sur des données indépendantes.
- **Pourquoi ça marche ?** Les deux bases auront des biais, mais **pas les mêmes**. Par exemple, la base de validation montrée dans le cours a plus de chiens, et les chats apparaissent cette fois sur des fonds clairs. Si l'algorithme s'était basé sur la règle erronée "fond sombre = chat", il échouera sur la base de test, ce qui est une bonne chose car cela révèle que l'algorithme n'est pas robuste.
- **Condition** : Les bases de validation et de test doivent idéalement être **indépendantes**.

4. Les Algorithmes Supervisés et l'Apprentissage

A. Qu'est-ce qu'un algorithme supervisé ?

- C'est un algorithme qui **apprend par lui-même à partir de grandes quantités de données**.
- **Exemple** : Les réseaux de neurones (qui seront vus plus tard dans le cours).
- C'est un algorithme qui **apprend à partir de données étiquetées** (chaque donnée est associée à une réponse attendue).
- Il apprend une fonction qui permet de **prédire la sortie à partir de l'entrée**.

B. Les Hyperparamètres

Bien que le modèle apprenne seul, le concepteur doit définir des choix de structure appelés **hyperparamètres**.

- *Exemple pour un réseau de neurones* : Le nombre de couches du réseau, ou le nombre de neurones.

C. La Base d'Apprentissage (ou d'Entraînement)

Dans le cas supervisé, on doit ajouter une troisième base de données pour permettre au modèle de s'entraîner.

5. Synthèse des Bases de Données Requises

Selon le type d'algorithme utilisé, voici les bases dont vous devez obligatoirement disposer :

1. Cas Non Supervisé (2 bases) :

- **Base de Validation** : Pour fixer/régler manuellement les paramètres.
- **Base de Test** : Pour évaluer les résultats finaux.

2. Cas Supervisé (3 bases) :

- **Base d'Apprentissage (ou d'Entraînement)** : Pour que l'algorithme apprenne de lui-même à partir des données.
- **Base de Validation** : Pour définir et régler les hyperparamètres (les choix d'architecture du modèle).
- **Base de Test** : Pour évaluer la capacité de généralisation et les performances finales avant de les annoncer au client.

6. La Règle d'Or de l'Évaluation

Pour conclure, que vous soyez dans un cas supervisé ou non supervisé, vous devez **connaître par cœur cette règle absolue** :

⚠ **On ne calcule les résultats sur le jeu (la base) de test qu'une seule et unique fois.**

Tous les développements, réglages et apprentissages doivent se faire uniquement sur les bases d'apprentissage et de validation. La base de test ne sert **strictement qu'à la fin du développement** pour mesurer la vraie performance de l'algorithme.

Cours : Les métriques pour évaluer un système de traitement des images

1. Introduction aux métriques et à la vérité terrain

L'objectif des **métriques** est de pouvoir annoncer un score de performance fiable pour un algorithme (ex: "mon algorithme fonctionne dans X % des cas"). Pour calculer ces métriques, on suit une règle fondamentale : on compare les **prédictions** de l'algorithme (le résultat fourni par la machine pour une image donnée) à la **vérité terrain**.

- **La vérité terrain** : Ce sont des étiquettes (ou *labels*) **définies manuellement par des humains**. Elles correspondent à la sortie idéale et parfaite que l'algorithme devrait théoriquement trouver.
- Ces étiquettes doivent être créées sur de grandes quantités d'images pour constituer la **base de test** (vue dans le cours précédent) et garantir des scores de performance fiables et représentatifs de la réalité.

2. Évaluer une tâche de classification binaire

La classification binaire consiste à classer une image entre deux catégories (ex: contient un chien ou ne contient pas de chien / contient un chat). Pour l'évaluer, on utilise un outil fondamental : la matrice de confusion.

A. La Matrice de Confusion

C'est un tableau à double entrée qui croise les prédictions de l'algorithme avec la vérité terrain. Pour la construire, on définit une classe "Positive" (ex: Chien) et une classe "Négative" (ex: Chat). Le tableau se remplit avec quatre valeurs :

- **Vrais Positifs (TP - True Positives)** : L'image contenait un chien (vérité), et l'algorithme a bien prédit un chien.
- **Faux Positifs (FP - False Positives)** : L'algorithme a prédit un chien, mais l'image contenait en réalité un chat.
- **Faux Négatifs (FN - False Negatives)** : L'algorithme a prédit un chat (négatif), alors que l'image contenait en réalité un chien (positif).
- **Vrais Négatifs (TN - True Negatives)** : L'image contenait un chat, et l'algorithme a bien prédit un chat.

B. Les Métriques dérivées de la matrice

À partir de ces quatre valeurs, on calcule des indicateurs clés, qui sont des ratios compris entre 0 et 1 :

1. La Précision :

- **Formule** : $\text{Vrais Positifs} / (\text{Vrais Positifs} + \text{Faux Positifs})$ (**Positifs**)
- **Interprétation** : Parmi toutes les fois où l'algorithme a affirmé "c'est un chien", dans quel pourcentage des cas avait-il réellement raison ? Cela mesure la fiabilité de la prédiction lorsqu'elle est déclenchée.

2. Le Rappel (Recall) :

- **Formule** : $\text{Vrais Positifs} / (\text{Vrais Positifs} + \text{Faux Négatifs})$ (**True Positifs**)
- **Interprétation** : Parmi tous les chiens qui existaient *réellement* dans la base de test, quel pourcentage l'algorithme a-t-il réussi à trouver ? Cela mesure la capacité du système à ne pas rater les éléments qu'il cherche.

3. Le Taux d'erreur :

- **Formule** : $(\text{Faux Positifs} + \text{Faux Négatifs}) / \text{Total des images}$ (**All**)
- **Interprétation** : La proportion totale d'erreurs commises par le système.

4. Le Score F1 :

- **Formule** : $2 * (\text{Précision} * \text{Rappel}) / (\text{Précision} + \text{Rappel})$
- **Explication** : C'est une moyenne **harmonique** (et non arithmétique) de la précision et du rappel.
- **Pourquoi une moyenne harmonique ?** Si un système à 100% de précision mais 0% de rappel (il ne prédit "chien" que quand il est absolument sûr, mais rate 99% des chiens existants), c'est un très mauvais système. Une moyenne arithmétique donnerait un score de 50%, ce qui est trompeur. La moyenne harmonique, à cause de la multiplication au numérateur, donnera un Score F1 de 0, ce qui reflète bien la nullité du système dans ce cas précis.

Note : Attention aux cas impossibles. Si un algorithme ne prédit **jamais** la classe "chien", le calcul de la précision provoque une division par zéro ($0 / 0+0$). La précision n'est donc pas définie (Not a Number), ce qui rend également le Score F1 indéfini.

3. Évaluer une tâche de régression

La régression ne cherche pas à deviner une catégorie, mais à **prédire un nombre réel** (ex: compter le nombre exact de chiens présents dans une image). Soit $\hat{Y}_i$ la prédiction de l'algorithme et Y_i la vérité terrain.

A. L'Erreur Absolue Moyenne (MAE - Mean Absolute Error)

- **Calcul** : On fait la somme des valeurs absolues des différences entre la prédiction et la vérité terrain, divisée par le nombre d'images (N).
- **Interprétation** : Elle renseigne de manière directe et linéaire sur la distance moyenne qui sépare nos prédictions de la vérité.

B. L'Erreur Quadratique Moyenne (MSE - Mean Squared Error)

- **Calcul** : On fait la somme des carrés des différences entre la prédiction et la vérité terrain, divisée par N.
- **Interprétation** : Parce qu'elle utilise le carré de l'erreur, la MSE va **pénaliser beaucoup plus lourdement les prédictions qui sont très éloignées de la vérité** par rapport à la MAE.

C. Exemple comparatif MAE vs MSE

Le cours prend l'exemple de 5 images contenant respectivement {2, 5, 4, 7, 3} chiens (Vérité terrain).

- **Système 1** : Prédit {2, 4, 4, 12, 3}.
 - Le système a souvent raison, mais se trompe de manière catastrophique sur l'image 4 (prédit 12 au lieu de 7).
 - $MAE = 6/5$.
 - $MSE = 26/5$.
- **Système 2** : Prédit {3, 4, 5, 5, 4}.
 - Le système se trompe sur toutes les images, mais de très peu à chaque fois.
 - $MAE = 6/5$ (La MAE est identique au Système 1 !).
 - $MSE = 8/5$.

Conclusion de l'exemple : Bien que les deux systèmes fassent le même nombre total d'erreurs en valeur absolue ($MAE = 6/5$), le Système 1 est lourdement pénalisé par la MSE ($26/5$ contre $8/5$) à cause de son erreur unique mais gigantesque (12 au lieu de 7), démontrant l'utilité de la MSE pour chasser les grosses aberrations.

Cours : Traitement des Images - Quantification et Résolution

Chapitre 1 : La Quantification

1.1. Définition et principe

La quantification définit la manière dont on encode l'information (les valeurs des niveaux de gris ou les couleurs) pour chaque pixel d'une image. Elle repose sur le nombre de bits alloués pour stocker cette information, allant généralement de 2 à 16 bits (ou plus).

- **Image binaire** : L'image est encodée de manière minimale (1 bit implicite) avec seulement deux états : noir ou blanc.
- **Augmentation des bits** : En augmentant le nombre de bits alloués, le nombre de valeurs possibles augmente de manière quadratique (nrmlmn c exponentielle parce que $c = \text{pow}(2, \text{nombre de bits})$). Par exemple, sur **2 bits**, on dispose de **4 valeurs** (ou 4 niveaux de gris) possibles.

1.2. Effet visuel du nombre de bits alloués

Lorsqu'on observe l'échelle de quantification :

- Avec un faible nombre de bits, on constate une frontière très ferme et abrupte entre les zones noires et blanches.
- Au fur et à mesure que le nombre de bits augmente, cette frontière s'estompe progressivement pour laisser place à des dégradés et des nuances subtiles.

Étude de cas visuelle : En partant d'une image couleur convertie en niveaux de gris :

- **Sur 32 niveaux** : Il y a relativement peu de différence visible à l'œil nu par rapport à l'image originale (qui est codée sur 255 niveaux).
- **Sur 16 niveaux, puis 8 niveaux (3 bits), 4 niveaux et enfin 2 niveaux (binaire)** : L'information visuelle se dégrade très rapidement.
- **Conclusion** : Plus on utilise de bits, plus on obtient de détails dans l'image. En contrepartie, le fichier devient inévitablement plus lourd.

1.3. Impact sur le poids du fichier (Calculs détaillés)

Le choix de la quantification influence linéairement la taille globale de l'image. Voici les calculs d'encombrement pour une image de **512 x 512 pixels** (sans aucune compression de l'information) :

- **Image binaire (1 bit par pixel)** : $512 \times 512 \times 1 \text{ bit} = 262\,144 \text{ bits}$. Divisé par 1024 = 256 Kilobits. Divisé par 8 = **32 Ko**.
- **Image sur 2 bits** : Le poids est multiplié par 2. $(512 \times 512 \times 2) / 1024 / 8 = \mathbf{64 \text{ Ko}}$.
- **Image sur 1 octet (Classique, valeurs de 0 à 255)** : **256 Ko**.
- **Image RGB (3 octets)** : Environ **758 Ko** (valeur mentionnée dans l'exemple).

Règles de conversion :

- **1 octet = 8 bits**
- Chaque unité suivante vaut $\times 1024$:
 - 1 Ko = 1024 octets
 - 1 Mo = 1024 Ko
 - 1 Go = 1024 Mo
 - 1 To = 1024 Go

1.4. Les Nombres Flottants et le comportement par défaut de Numpy

Les images peuvent également être encodées sur des nombres flottants plutôt que sur des entiers.

- **Flottant sur un système 64 bits (image en niveaux de gris)** : L'image pèsera environ **2 Mo**.
- **Flottant sur un système 64 bits (image RGB)** : L'image pèsera environ **6 Mo**.
- **float64 = 64 bits = 8 octets par valeur**

Point d'attention pratique (Travaux Dirigés) : Ce codage lourd en nombres flottants correspond au comportement par défaut de la bibliothèque **Numpy**. En TD, si vous créez une image sans préciser explicitement le type désiré, votre image pèsera directement 2 Mo ou 6 Mo (selon le nombre de canaux utilisés).

Chapitre 2 : L'Échantillonnage et la Résolution

2.1. Distinction fondamentale

L'échantillonnage est le processus qui influence la résolution de l'image. **Attention :** Il ne faut jamais confondre la résolution et la taille de l'image. La résolution correspond à l'information spatiale que l'on est physiquement capable de distinguer.

2.2. La Résolution Spatiale : Notion de "Distance d'échantillonnage au sol"

L'exemple d'une image aérienne permet de bien visualiser ce concept :

- **Résolution de 250 mètres :** Il est impossible de différencier des objets mesurant moins de 250 m. Un tel objet est entièrement contenu dans un seul pixel.
- **Résolution de 60 mètres :** Des formes globales commencent à apparaître.
- **Résolution de 10 mètres :** L'origine de l'image devient évidente. On distingue les bâtiments, la Seine, ainsi que des lieux célèbres (le Louvre, les Tuileries). Les objets faisant au moins 10 m (comme la largeur des ponts) apparaissent clairement.
- **Résolution de 20 centimètres :** L'image regorge de détails très nets (par exemple, on distingue parfaitement les voitures).

2.3. Les opérations d'échantillonnage (Upsampling et Downsampling)

- **Upsampling (Suréchantillonnage) :** Consiste à rendre l'échantillonnage plus fin pour augmenter la résolution.
 - **De manière naïve :** Il est physiquement impossible d'ajouter de la nouvelle information, car l'information originale a été figée au moment de l'acquisition photographique.
 - **Via des méthodes avancées (Reconnaissance de formes / IA) :** Des algorithmes (utilisant notamment des réseaux de neurones) peuvent deviner et interpoler des détails. Par exemple, si le système reconnaît la forme d'un cercle pixélisé, il reconstruira un cercle net lors du suréchantillonnage. Ces méthodes neuronales sont bien plus performantes et nettes que les techniques classiques (comme la déconvolution, qui a tendance à générer du flou et des artefacts visuels).
- **Downsampling (Sous-échantillonnage) :** Consiste à réduire la résolution de l'image.
 - Cette opération engendre obligatoirement une **perte d'information définitive** si l'on ne conserve pas une copie de l'image originale.

2.4. Impact de l'échantillonnage sur le poids du fichier

Contrairement à la quantification qui a un impact linéaire, l'échantillonnage a une relation **quadratique** par rapport au côté de l'image (pour des images de forme carrée).

Exemple de calcul pour une image RGB (1 octet par canal, soit 3 octets par pixel) :

- **32 x 32 pixels :** $32 \times 32 \times 3 \text{ octets} \approx 3 \text{ Ko}$.
- **64 x 64 pixels :** $64 \times 64 \times 3 \text{ octets} \approx 12 \text{ Ko}$. (Remarque : En multipliant le côté par 2, on multiplie la taille totale du fichier par 4).
- **2048 x 2048 pixels (env. 4 mégapixels) :** $\approx 12 \text{ Mo}$.

Chapitre 3 : Conséquences de l'Échantillonnage sur le Traitement des Images

3.1. Mesure et extraction d'information

L'échantillonnage a un effet direct sur ce qui est mesurable au sein d'une image, puisque c'est lui qui dicte la précision des données encodées.

Cas pratique : Estimation de l'aire d'un rectangle blanc par comptage de pixels :

- **Échantillonnage très fin (tendant vers l'infini) :** L'erreur d'estimation est extrêmement faible,

de l'ordre de **0,93 %**.

- **Échantillonnage réduit (64 x 64 pixels)** : Sur les frontières de l'objet, le système est contraint de prendre une décision binaire (le pixel limite est-il considéré noir ou blanc ?). L'erreur monte à **1,45 %**.
- **Échantillonnage à 32 x 32 pixels** : Les pixels deviennent trop gros. Dans les zones limites (haut et bas du rectangle), le blanc étant majoritaire à l'intérieur du pixel, l'intégralité du pixel est assignée à la valeur blanche. Les fines bordures noires sont perdues et l'erreur de calcul augmente.
- **Échantillonnage à 16 x 16 pixels** : Le comportement devient différent et imprévisible, particulièrement au niveau de l'arrondi des coins de l'objet.
- **Échantillonnage à 4 x 4 pixels** : Les bordures noires en haut, en bas et à gauche du rectangle ont totalement disparu du modèle. L'erreur d'estimation explose fortement.

3.2. Conclusion de la chaîne de traitement

Les choix en matière de **quantification** et de **résolution** sont décisifs. Ils doivent impérativement être adaptés aux objectifs spécifiques de votre chaîne de traitement d'images :

1. Moins vous conservez d'informations au départ, moins vous aurez de liberté pour effectuer des traitements poussés par la suite.
2. Face à un problème de traitement d'image, la question systématique à se poser est : "**Quelle quantité d'information et quel type de détails ai-je réellement besoin de conserver pour résoudre ce problème ?**"

Cours : Traitement des Images - Les Espaces de Couleurs

1. L'espace RGB (Rouge, Vert, Bleu)

C'est l'espace de couleur que l'on rencontre le plus couramment, notamment lors des premiers cours sur le traitement d'images.

- **Signification** : RGB signifie *Red, Green, Blue* (Rouge, Vert, Bleu).
- **Principe** : Il s'agit d'un espace de couleur **additif**. Pour créer une couleur donnée, on spécifie la quantité de chaque couleur primaire (rouge, vert, bleu) à additionner. C'est un principe similaire au mélange de peinture.
- **Application principale** : **Les écrans**. Cet espace est utilisé car il correspond à la conception physique des écrans : chaque pixel est composé de trois sources lumineuses (une rouge, une verte et une bleue) qui s'additionnent pour former l'image finale.

2. L'espace CMYK / CMJN (Cyan, Magenta, Jaune, Noir)

- **Signification** : CMYK signifie *Cyan, Magenta, Yellow, Black* (Cyan, Magenta, Jaune, Noir).
- **Principe** : Contrairement au RGB, c'est un espace de couleur **soustractif**. Au lieu d'ajouter de la lumière, on spécifie quelle quantité d'encre doit être appliquée pour *absorber* la lumière et ainsi restituer la couleur désirée.
- **Application principale** : **L'imprimerie**. C'est l'espace de couleur utilisé par les imprimantes couleurs, qui fonctionnent en superposant des encres (ou des toners) cyan, magenta, jaune et noire.

3. L'espace HSV (Teinte, Saturation, Valeur)

- **Signification** : HSV signifie *Hue, Saturation, Value* (Teinte, Saturation, Valeur).
 - **Teinte (Hue)** : Représente la couleur pure elle-même.
 - **Saturation** : Indique la force ou l'intensité de cette couleur.
 - **Valeur (Value)** : Représente la clarté.
- **Principe** : Cet espace permet de sélectionner une couleur d'une manière beaucoup plus intuitive et proche de la façon dont **l'œil humain perçoit naturellement les couleurs**, par rapport aux modèles RGB ou CMYK.

4. L'espace HSL (Teinte, Saturation, Luminosité)

- **Signification** : HSL signifie *Hue, Saturation, Lightness* (Teinte, Saturation, Luminosité).
- **Principe** : Tout comme le HSV, c'est une représentation très naturelle pour l'humain.
- **Application principale : La robustesse aux changements d'éclairage**. Lorsqu'on prend une photo, un changement de luminosité modifie fortement la représentation de l'image. En traitement d'images, on souhaite souvent extraire des informations (les objets, les formes) sans être perturbé par l'éclairage. L'espace HSL permet de s'abstraire de la luminosité (le canal L) pour se concentrer uniquement sur l'information utile contenue dans la teinte et la saturation.

Note sur les conversions : Il est tout à fait possible de passer de n'importe quel espace de couleur à un autre. Il existe pour cela des formules mathématiques spécifiques (plus ou moins complexes) qui permettent ces transformations.

5. La conversion d'une image Couleur vers un Niveau de Gris

Passer d'une image couleur (codée sur 3 canaux, par exemple RGB) à une image en niveau de gris (codée sur 1 seul canal) implique inévitablement une **perte d'information**. Il existe trois méthodes principales pour effectuer cette conversion :

Méthode A : Ne garder qu'un seul canal

- **Principe** : On supprime deux canaux sur trois (ex: on ne garde que le canal Bleu d'une image RGB).
- **Résultat** : C'est généralement une mauvaise représentation de l'image originale. *Exemple : si l'on ne garde que le canal bleu sur une image contenant un poivron rouge et un poivron vert, il sera très difficile de les distinguer.*

Méthode B : Faire la moyenne des trois canaux

- **Principe** : Pour chaque pixel, on calcule la moyenne mathématique des valeurs : (Rouge + Vert + Bleu) / 3.
- **Résultat** : L'image est bien meilleure qu'avec un seul canal, et les différences de couleurs (comme entre les poivrons) sont nettement plus visibles.

Méthode C : Utiliser la formule de Luminance (La méthode recommandée)

- **Principe** : On utilise une formule spécifique définie par un standard, basée sur des coefficients particuliers appliqués à chaque canal. Cette formule a été déterminée suite à des tests sur la perception visuelle humaine (c'est d'ailleurs cette formule qui permet d'obtenir la "Luminosité" dans l'espace HSL).
- **Résultat** : C'est la méthode à privilégier. Elle donne l'image la plus agréable visuellement et représente le plus fidèlement les véritables variations de luminosité perçues par l'œil humain.
- **Formule** : $Y = 0.299 * R + 0.587 * G + 0.114 * B$

6. L'inversion des couleurs

Il existe de nombreuses opérations possibles sur les espaces de couleurs. L'une des plus classiques est l'**inversion**, souvent utilisée pour créer des filtres (comme ceux des téléphones portables).

- **Principe** : Cette opération peut être réalisée dans n'importe quel espace colorimétrique, donnant à chaque fois un rendu différent.
- **Exemple dans l'espace RGB (sur 8 bits)** : Si l'image est codée sur des entiers de 8 bits (valeurs de 0 à 255), on inverse mathématiquement la valeur de chaque pixel pour chaque canal de la façon suivante :
 - Nouveau Rouge = 255 - Valeur du Rouge actuel
 - Nouveau Vert = 255 - Valeur du Vert actuel
 - Nouveau Bleu = 255 - Valeur du Bleu actuel
- **Résultat** : On obtient une image aux couleurs inversées (semblable à un négatif photographique).

Cours : Traitement des Images — Le Seuillage

Introduction : Les opérations ponctuelles

Une **opération ponctuelle** est une opération appliquée **pixel par pixel**.

- Chaque pixel est traité **indépendamment**.
- La nouvelle valeur d'un pixel dépend **uniquement de sa valeur actuelle**.
- Les pixels voisins **ne sont pas utilisés** dans le calcul.

Une des opérations ponctuelles les plus importantes est le **seuillage**.

1. Le concept de seuillage

1.1 Principe de base

Le **seuillage** consiste à choisir une **valeur seuil** et à comparer chaque pixel de l'image à cette valeur.

Pour chaque pixel :

- Si la valeur du pixel $\leq$ **seuil** $\rightarrow$ pixel = **0** (noir)
- Si la valeur du pixel $>$ **seuil** $\rightarrow$ pixel = **1** (blanc)

Résultat

L'image obtenue devient une **image binaire** :

- pixels noirs = 0
- pixels blancs = 1

Cette image montre **les zones en dessous ou au-dessus du seuil**.

1.2 Formalisation mathématique

On considère une image comme une fonction :

$$I : \mathbb{N}^2 \rightarrow \mathbb{N}$$

Pour un pixel situé en **(i, j)** :

Si $\text{Pixel_entrée}(i,j) > \text{seuil}$

$\rightarrow \text{Pixel_sortie}(i,j) = 1$

Sinon

$\rightarrow \text{Pixel_sortie}(i,j) = 0$

Donc :

$\text{Pixel_sortie}(i,j) = 1$ si $\text{Pixel_entrée}(i,j) > \text{seuil}$

$\text{Pixel_sortie}(i,j) = 0$ si $\text{Pixel_entrée}(i,j) \leq \text{seuil}$

Remarque :

Dans certains cas, on peut inverser les valeurs (0 et 1).

Le principe reste simplement de **séparer les pixels en deux groupes**.

1.3 Variante : le seuillage par hystérésis

Le **seuillage par hystérésis** utilise **deux seuils** :

- un **seuil bas**
- un **seuil haut**

Les pixels sont alors classés en trois catégories :

- pixel $<$ seuil bas
- pixel entre les deux seuils
- pixel $>$ seuil haut

Cette méthode est souvent utilisée dans la **détection de contours**.

2. Utilité du seuillage : exemple pratique

Le seuillage permet de **séparer un objet du fond**.

Exemple : image d'un chat

Étapes :

1. L'image couleur est transformée en **niveaux de gris**.
2. On applique un **seuil de 70**.
3. Les pixels **inférieurs à 70** (zones sombres du chat) deviennent **1**.
4. Les pixels **supérieurs à 70** (fond clair) deviennent **0**.

Résultat

Le chat est **sélectionné automatiquement** dans l'image.

Cette étape est importante pour :

- détecter des objets
- compter des objets
- isoler un élément dans une image

Problème

Comment choisir la **bonne valeur de seuil** ?

3. L'outil fondamental : l'histogramme

3.1 Définition

Un **histogramme** représente la **distribution des intensités des pixels** dans une image.

Pour une image en niveaux de gris :

- les valeurs possibles vont de **0 à 255**
- on compte **combien de pixels possèdent chaque valeur**

L'histogramme montre donc la **fréquence de chaque intensité**.

3.2 Construction de l'histogramme

Étapes :

1. On crée des **cases (bins)** pour les intensités.
2. L'ordinateur parcourt tous les pixels de l'image.
3. Pour chaque pixel, on **incrémente la case correspondante**.

Exemple :

Si un pixel vaut **90**, on ajoute **+1** dans la case 90.

À la fin, on obtient un **graphique** qui montre :

- les valeurs fréquentes
- les valeurs rares

Les **pics** de l'histogramme correspondent aux **intensités dominantes**.

3.3 Propriété importante : perte d'information spatiale

L'histogramme **ne contient pas d'information sur la position des pixels**.

Il indique uniquement **combien de pixels ont telle valeur**, mais pas **où ils se trouvent**.

Conséquence :

Deux images **visuellement différentes** peuvent avoir le **même histogramme** si leurs pixels ont la même distribution.

4. Choisir un seuil grâce à l'histogramme

4.1 Histogramme comme mélange de gaussiennes

Dans beaucoup d'images, l'histogramme possède **plusieurs pics**.

Par exemple :

- un pic pour **l'objet**
- un pic pour **le fond**

Mathématiquement, on peut modéliser cela comme un **mélange de distributions gaussiennes**.

Exemple :

- pixels sombres (chat) → valeurs faibles

- pixels clairs (fond) → valeurs élevées

4.2 Le problème du chevauchement

Dans la réalité, les deux distributions **se chevauchent**.

Cela signifie que :

- certains pixels du fond ressemblent à l'objet
- certains pixels de l'objet ressemblent au fond

Donc **aucun seuil ne sépare parfaitement les deux classes**.

Il y aura toujours :

- un peu de fond sélectionné
- ou une partie de l'objet perdue

4.3 Méthodes pour choisir le seuil

1. Choix empirique

Tester plusieurs valeurs et regarder le résultat.

Inconvénient : méthode **peu efficace**.

2. Analyse visuelle de l'histogramme

On observe l'histogramme :

- on repère les **deux pics**
- on place le seuil **dans la vallée entre les deux**

Exemple : seuil = **70** pour l'image du chat.

3. Automatisation

Dans les applications réelles, il est impossible de choisir le seuil manuellement.

On utilise donc **des algorithmes automatiques**.

4. Algorithme d'Otsu

L'algorithme **d'Otsu** permet de **calculer automatiquement le seuil optimal** à partir de l'histogramme.

Il est très utilisé dans :

- vision par ordinateur
- segmentation d'images
- traitement automatique d'images

Conclusion

Le **seuillage** est une opération simple mais fondamentale en traitement d'images.

Il permet de :

- transformer une image en **image binaire**
- **séparer un objet du fond**
- préparer les étapes de **segmentation et d'analyse**

Le choix du seuil peut être :

- manuel
- basé sur l'histogramme
- ou **calculé automatiquement** (ex : méthode d'Otsu).

Cours : Traitement des Images — Clustering et Algorithme d'Otsu

1. Introduction et contexte

En traitement d'images, on cherche souvent à **séparer un objet de son fond**.

Pour cela, on utilise **le seuillage**.

Une image peut être analysée grâce à **son histogramme**, qui montre la distribution des valeurs des pixels.

Souvent, cet histogramme peut être modélisé comme un **mélange de deux distributions gaussiennes** :

- une gaussienne pour **l'objet**
- une gaussienne pour **le fond**

Le problème est de **trouver un seuil optimal** pour séparer ces deux groupes de pixels.

2. Le problème du seuillage manuel

Les méthodes classiques consistent à :

- choisir un seuil **au hasard**
- choisir un seuil **visuellement sur l'histogramme**

Ces méthodes nécessitent **une intervention humaine**, ce qui est impossible dans les systèmes automatiques.

Exemples d'applications automatiques :

- détournage d'objets dans un smartphone
- vision par ordinateur
- analyse automatique d'images

3. La solution : l'algorithme d'Otsu

L'**algorithme d'Otsu** (1979) permet de **déterminer automatiquement le meilleur seuil**.

Objectif :

Trouver le seuil qui **sépare le mieux les pixels en deux classes**.

Remarque importante :

Si les distributions **se chevauchent**, la séparation **ne sera jamais parfaite**.

4. Hypothèses de l'algorithme d'Otsu

L'algorithme repose sur trois hypothèses importantes.

1. Fond uniforme

Les pixels du fond ont **des valeurs proches entre elles**.

2. Objet uniforme

Les pixels de l'objet ont également **des valeurs similaires**.

3. Distributions relativement séparées

Cela implique deux propriétés statistiques :

Variance intra-classe faible

- les valeurs d'une même classe sont proches

Variance inter-classe élevée

- les moyennes des deux classes sont éloignées

5. Principe de l'algorithme d'Otsu

Le principe mathématique est :

Minimiser la variance intra-classe

équivalent à

Maximiser la variance inter-classe.

Autrement dit :

On cherche un seuil qui rend **chaque classe la plus homogène possible**.

Fonctionnement de l'algorithme

1. L'algorithme teste **tous les seuils possibles**.
2. Pour chaque seuil, il divise les pixels en **deux classes**.
3. Il calcule la **variance intra-classe (globale)** en suivant des étapes prédéfinies.
4. Il choisit le seuil qui donne la **variance intra-classe minimale**.

Ce seuil est considéré comme le **seuil optimal**.

6. Calcul de la variance intra-classe

Pour un seuil donné, on obtient deux classes :

- Classe 1
- Classe 2

Le calcul nécessite plusieurs éléments.

Probabilité d'appartenir à la classe 1

P_1 = pixels classe 1 / pixels totaux

Probabilité d'appartenir à la classe 2

P_2 = pixels classe 2 / pixels totaux

Variance intra-classe de la classe 1

On calcule :

1. la moyenne μ_1 des pixels
2. la moyenne des carrés des écarts à la moyenne, on obtient la variance intra classe 1 : σ_1^2

Variance intra-classe de la classe 2

Même calcul pour la classe 2 :

Variance intra-classe globale

La variance intra-classe globale est :

$$\sigma^2 = P_1 \sigma_1^2 + P_2 \sigma_2^2$$

Plus cette valeur est **petite**, meilleure est la séparation.

7. Clustering vs segmentation sémantique

Il faut distinguer deux concepts importants.

Clustering (comme Otsu)

Le clustering :

- regroupe les pixels **selon leurs valeurs**
- crée des **classes mathématiques**

Mais l'algorithme **ne sait pas ce que représentent ces classes**.

Exemple :

- Classe 1
- Classe 2

C'est à l'humain d'interpréter :

- Classe sombre = objet
- Classe claire = fond

Segmentation sémantique

La segmentation sémantique est beaucoup plus avancée.

Elle attribue **un sens réel à chaque pixel**.

Exemple :

- pixel = chat
- pixel = arbre
- pixel = route

Elle utilise souvent l'intelligence artificielle.

8. Algorithme d'Otsu multi-classes

L'algorithme peut être étendu pour créer **plusieurs classes**.

Exemple :

3 classes avec **2 seuils**.

Règle :

- pixels < seuil 1 → **classe 1**
- seuil 1 ≤ pixels < seuil 2 → **classe 2**
- pixels ≥ seuil 2 → **classe 3**

Problème : explosion combinatoire

Pour une image 8 bits :

- 256 valeurs possibles

Pour deux seuils :

256 × 256 combinaisons possibles.

Donc le temps de calcul augmente **très rapidement**.

Des optimisations sont nécessaires pour rendre l'algorithme efficace.

9. Exemple de calcul (image 3×3)

Considérons l'image suivante :

3, 3, 5

4, 5, 8

8, 12, 12

Nombre total de pixels : 9

Test 1 : seuil = 6

Classe 1 (<6) :

3, 3, 4, 5, 5

Nombre de pixels = 5

$P_1 = 5 / 9$

Moyenne :

$\mu_1 = 4$

Variance :

$\sigma_1^2 = 4/5$

Classe 2 (>6) :

8, 8, 12, 12

Nombre de pixels = 4

$P_2 = 4 / 9$

Moyenne :

$\mu_2 = 10$

Variance :

$\sigma_2^2 = 4$

Score Otsu :

$\sigma^2 \approx 2.22$

Test 2 : seuil = 9

Classe 1 (<9) :

3, 3, 4, 5, 5, 8, 8

$P_1 = 7 / 9$

Moyenne :

$\mu_1 \approx 5.14$

Variance :

$\sigma_1^2 \approx 3.84$

Classe 2 (>9) :

12, 12

$P_2 = 2 / 9$

Moyenne :

$\mu_2 = 12$

Variance :

$\sigma_2^2 = 0$

Score Otsu :

$\sigma^2 \approx 2.98$

10. Conclusion de l'exemple

Comparaison :

- seuil 6 $\rightarrow$ score ≈ 2.22
- seuil 9 $\rightarrow$ score ≈ 2.98

L'algorithme choisit **le seuil 6** car il donne **la variance intra-classe la plus faible**.

Donc la séparation des pixels est **meilleure**.

Conclusion générale

L'algorithme d'Otsu est une méthode automatique de seuillage qui :

- analyse **l'histogramme**
- teste **tous les seuils possibles**
- choisit celui qui **minimise la variance intra-classe**

Il est largement utilisé en :

- vision par ordinateur
- segmentation d'images
- traitement automatique d'images.

Cours : Traitement des Images - Clustering avec l'algorithme des K-moyennes

Introduction

Dans le module précédent, nous avons vu l'algorithme d'Otsu pour séparer les pixels d'une image. Dans ce module, nous abordons un deuxième algorithme de clustering fondamental : **les K-moyennes** (ou *K-means* en anglais).

L'objectif reste le même : diviser notre image en groupes de pixels (clusters) partageant des caractéristiques similaires.

1. Principe de base des K-moyennes

1.1. Modélisation par Centroïdes

Contrairement à certaines méthodes qui modélisent un groupe de pixels par une **distribution globale** (par exemple une gaussienne), l'algorithme des **K-moyennes représente chaque cluster par un point central appelé centroïde**.

- **K** représente le nombre de clusters que l'on souhaite obtenir.

Exemples :

- **K=2**
- **K=3**
- **K=4**

Le **centroïde** est simplement la **valeur moyenne des pixels du cluster**, et il sert de **représentant du groupe**.

1.2. Différence avec la segmentation sémantique

Comme l'algorithme d'Otsu, les K-moyennes réalisent **du clustering et non de la segmentation sémantique**.

Cela signifie que l'algorithme :

- **regroupe les pixels selon leurs valeurs**
- **ne comprend pas ce que représentent les pixels**

Exemple :

- un cluster peut correspondre au **chat**
- un autre au **fond**

Mais l'algorithme **ne sait pas que c'est un chat**.

Il ne produit que **des classes mathématiques**.

De plus, les **valeurs ou couleurs finales peuvent être inversées** par rapport à l'image d'origine.

2. Déroulement de l'algorithme

L'algorithme est **itératif** et se déroule en **trois étapes principales**.

Étape 1 : Initialisation

On commence par **définir les valeurs initiales des K centroïdes**.

Cette initialisation peut être faite :

- par des **heuristiques**
- ou **aléatoirement**

Dans la pratique, on choisit souvent **K valeurs au hasard dans l'image**.

Étape 2 : Assignment des pixels

Pour les centroïdes fixés à l'étape actuelle :

1. on parcourt **chaque pixel de l'image**
2. on calcule la **distance entre ce pixel et chaque centroïde**

Règle :

Le pixel est assigné au cluster dont le centroïde est le plus proche.

Étape 3 : Mise à jour des centroïdes

Une fois tous les pixels assignés :

- on **recalcule chaque centroïde**

Règle :

Le nouveau centroïde est la moyenne des pixels appartenant à son cluster.

Répétition du processus

On répète ensuite :

- **Étape 2 : assignment**
- **Étape 3 : mise à jour**

jusqu'à ce que l'algorithme **converge**.

La Convergence

L'algorithme a **convergé** lorsque :

- aucun pixel ne change de cluster entre deux itérations
- les **centroïdes ne changent plus**

À ce moment-là, **l'algorithme s'arrête**.

3. Exemple d'application détaillé

Considérons une image simplifiée **3×3 (9 pixels)** :

3, 3, 4, 5, 5, 8, 8, 12, 12

Nous voulons **K = 2 clusters**.

3.1 Test avec une initialisation à 3 et 6

Centroïdes initiaux :

- **C1 = 3**
- **C2 = 6**

Itération 1 :

Assignment

Classe 1 :

3, 3, 4 (plus proches de 3)

Classe 2 :

5, 5, 8, 8, 12, 12 (plus proches de 6)

Mise à jour des centroïdes

C1 = (3+3+4)/3=3.33

$$C2 = (5+5+8+8+12+12)/6=8.33$$

Itération 2 :

Nouveaux centres :

- 3.33
- 8.33

Assignation :

Classe 1 :

3, 3, 4, 5, 5

Classe 2 :

8, 8, 12, 12

(les pixels 5 changent de cluster)

Nouveaux centroïdes

$$C1 = (3+3+4+5+5)/5=4$$

$$C2 = (8+8+12+12)/4=10$$

Itération 3 :

Centres :

- 4
- 10

Assignation :

Classe 1 :

3, 3, 4, 5, 5

Classe 2 :

8, 8, 12, 12

Aucun pixel ne change.

L'algorithme a convergé.

Centres finaux :

- **C1** = 4
- **C2** = 10

Conclusion sur l'initialisation aléatoire

Dans cet exemple simple :

deux initialisations différentes donnent le même résultat final.

Cependant, dans des images **plus grandes et plus complexes** :

- l'initialisation peut **changer légèrement le résultat**
- elle peut **influencer le nombre d'itérations nécessaires**

4. Comparaison : K-moyennes vs Otsu

Critère	Otsu	K-means
Nombre de classes	Arbitraire	Arbitraire
Itérations	Nombre de seuils possibles (fixe)	Dépend de l'initialisation
Vitesse pour plusieurs classes	Peut devenir lent	Plus rapide
Optimal	Oui (teste toutes les solutions)	Non (minimum local possible)

Conclusion

Le choix dépend des contraintes :

Utiliser K-moyennes si :

- beaucoup de classes
- contraintes de temps
- images grandes

Utiliser Otsu si :

- peu de classes
- besoin d'une **solution mathématiquement optimale**
- temps de calcul moins critique.

Cours : Traitement des Images — Opérations Ponctuelles (Égalisation, Normalisation et autres)

Introduction aux Opérations Ponctuelles

Dans les modules précédents, nous avons abordé le **seuillage**. Ce cours se concentre sur d'autres **opérations ponctuelles**, c'est-à-dire des transformations qui s'appliquent **individuellement à chaque pixel de l'image**, indépendamment de ses voisins.

Ces opérations sont fondamentales pour **préparer et nettoyer les images** avant des traitements plus complexes.

1. L'Égalisation d'Histogramme

1.1. Le problème de la dynamique faible

Lorsqu'une image est **très sombre** (comme l'exemple du chat présenté dans le cours), son **histogramme est tassé vers la gauche**, c'est-à-dire vers les **valeurs faibles** (proches de 0, le noir).

La plage de valeurs possibles (**0 à 255 pour une image 8 bits**) est donc **sous-utilisée**.

On dit que l'image a **une dynamique faible**, ce qui rend les **détails difficiles à distinguer**.

1.2. Objectif de l'égalisation

Le but de l'**égalisation d'histogramme** est d'**augmenter la dynamique de l'image**.

Pour cela, on cherche à :

- **étaler l'histogramme**
- **utiliser toute la plage 0 à 255**
- **tendre vers une distribution uniforme des pixels**

Mathématiquement :

l'**histogramme cumulé** de la nouvelle image doit devenir **linéaire**.

1.3. Modélisation mathématique

1- Histogramme original H_I

Il vit dans un **espace à 256 dimensions** (image 8 bits).

Chaque dimension représente le **nombre de pixels pour une intensité donnée**.

2- Histogramme cumulé normalisé $C_I(i)$

Pour une intensité i (ex : 3) :

1. on somme **tous les pixels de valeur $\leq i$**
2. on divise par le **nombre total de pixels**

On obtient donc la **proportion de pixels $\leq i$** .

3- Transformation T

On cherche une transformation $T : x \rightarrow y$ qui projette l'image $I \rightarrow J$:

de manière à ce que l'**histogramme cumulé de J soit linéaire** :

$$C_J(i) = (i+1) \times k$$

4- Formule finale

La nouvelle valeur y d'un pixel de valeur initiale x est :

$$y = \max(0, 256 \times C_I(x) - 1)$$

la fonction **$\max(0, \dots)$** évite les **valeurs négatives**.

1.4. Application aux images couleurs (RGB)

Si on applique cette transformation **séparément sur R, G et B**, on peut **modifier les couleurs de l'image**, car la transformation **n'est pas linéaire**.

La bonne méthode :

1. Convertir l'image **RGB $\rightarrow$ HSV**

2. Appliquer l'égalisation **uniquement** sur le canal V (Valeur / intensité)
3. Reconvertir **HSV → RGB**

2. Opérations Arithmétiques Simples

(Addition et Soustraction de constantes)

On peut modifier l'histogramme en **ajoutant ou en soustrayant une constante** à chaque pixel.

Addition d'une constante (ex : +50)

Effet :

- l'histogramme se **décale vers la droite**
- l'image **s'éclaircit**

Soustraction d'une constante

Effet :

- l'histogramme se **décale vers la gauche**
- l'image **s'assombrit**

3. La Normalisation d'Image

La **normalisation** consiste à :

- **centrer**
- **réduire**

les valeurs d'une image en utilisant :

- la **moyenne**
- l'**écart-type**

Pourquoi normaliser ?

Dans de nombreux algorithmes (notamment en **Machine Learning**) :

- la **moyenne globale**
- la **variance absolue**

ne sont **pas des informations pertinentes**.

Exemple :

Si on ajoute **+50 à tous les pixels**, l'image a :

- une **moyenne différente**
- mais **exactement la même structure**

La normalisation permet donc aux algorithmes de se concentrer sur :

- la **distribution des pixels**
- plutôt que sur l'**illumination globale**.

$I_{\text{norm}}(i) = (I(i) - \text{moyenne}) / \text{l'écart-type}$

4. La Multiplication d'Images (Masquage)

La multiplication se fait **pixel par pixel entre deux images**.

Elle est surtout utilisée pour le **masquage**.

Le masque est une **image binaire** composée de :

- **0**
- **1**

souvent obtenue après un **seuillage**.

Principe :

- Pixel × **1** → la valeur est **conservée**
- Pixel × **0** → devient **noir**

Résultat :

On peut **isoler un objet** dans une image.

Exemple :

- garder **le chat**
- supprimer **le fond**

5. Réduction de Bruit par Moyennage

Cette opération permet de **nettoyer une image bruitée**.

Hypothèse

L'image est perturbée par un **bruit qui suit une loi normale centrée** :

Théorie mathématique

La somme de **N distributions normales centrées** réduit l'écart-type :

Application pratique (réduire l'écart-type du bruit)

Si on possède **plusieurs images bruitées d'une même scène** :

on prend **plusieurs photos de la même scène** (exactement la même image) mais chaque photo contient un **bruit différent**.

on peut **faire la moyenne des images**.

Résultat :

- **l'image réelle** → reste la même
- **le bruit** → diminue

6. Soustraction d'Images

La soustraction de deux images (souvent avec **valeur absolue**) permet de **mettre en évidence les différences**.

6.1 Vision industrielle (Contrôle qualité)

Principe :

1. on prend une **image parfaite de référence**
2. on la soustrait à l'image produite

Les défauts apparaissent immédiatement :

- connexions manquantes
- excès de matière
- erreurs de gravure

6.2 Soustraction de fond dans une vidéo

Technique utilisée pour **détecter les mouvements**.

Étape 1 (init -> une seule fois)

La **première image** devient l'**image de fond**.

Fond = Image_1

Repete :

Étape 2 : détection du mouvement

On soustrait :

- **l'image nouvelle**
- **l'image de fond**

Résultat :

- les éléments **statiques disparaissent**
- seuls les **objets en mouvement restent visibles**

Étape 3 : mise à jour du fond

Le fond est mis à jour avec une **moyenne pondérée** :

Fond = $(\alpha \times \text{Fond}) + ((1 - \alpha) \times \text{Image_nouvelle})$

où :

- α est souvent **élevé** (ex : 0.95)
- cela permet de **garder une mémoire temporelle du fond**

Until end of the video

Exemple : les yeux d'un chat dans l'obscurité.

Conclusion

Les **opérations ponctuelles** sont mathématiquement **simples**, car elles agissent **pixel par pixel**. Cependant, elles constituent **la première étape essentielle du traitement d'images (pré-processing)** :

- réduction du bruit
- isolation d'objets
- correction d'éclairage
- préparation des données

Ces opérations sont donc **indispensables avant l'application d'algorithmes plus complexes d'extraction de caractéristiques**.

Cours : Traitement des Images - Opérations Locales et Convolution :

1. Introduction : L'importance du contexte

Jusqu'à ce stade du cours, les traitements d'image s'appuyaient sur des **opérations ponctuelles**, c'est-à-dire des opérations qui traitent les pixels de manière isolée, un par un.

- **La limite des opérations ponctuelles** : Isolé, un pixel ne donne aucune information sur le contenu global. Le professeur donne l'exemple d'un chat : en ne regardant qu'un seul pixel de l'image, il est strictement impossible de deviner ce qu'elle représente.
- **La solution (Les opérations locales)** : Pour pouvoir comprendre le contenu d'une image (ex: reconnaître un objet), il est impératif de prendre en compte le **contexte**. L'idée fondamentale des opérations locales est de ne plus analyser un pixel seul, mais de regarder **un pixel et son voisinage**.

2. Définition formelle d'une opération locale

Une opération locale détermine la nouvelle valeur d'un pixel en fonction des pixels qui l'entourent. Mathématiquement, cela se définit par une fonction f . Pour une image d'entrée I_i et une image de sortie I_o , la valeur du pixel aux coordonnées (i, j) dépend de son voisinage $N(i, j)$:

— Idée d'une opération locale:

La valeur du pixel (i, j) dans l'image résultat I_o dépend de son voisinage dans I_i :

$$I_o(i, j) = f\left(\left\{I_i(i', j') \mid \forall (i', j') \in \mathcal{N}_{i,j}\right\}\right)$$

Les différents types de voisinages (N)

Pour appliquer cette fonction, on utilise classiquement **plusieurs types de voisinages** (souvent illustrés sur des matrices (3×3) , bien qu'ils puissent être plus vastes) :

- **La 4-connexité** : On ne considère que le pixel central et ses 4 voisins directs en forme de croix (au-dessus, en dessous, à gauche et à droite).
- **La 8-connexité** : On élargit le voisinage en incluant les pixels situés dans les diagonales (le centre + 8 pixels autour).
- **Voisinages arbitraires** : Il est tout à fait possible de définir des géométries de voisinage sur-mesure selon les besoins spécifiques du traitement.

3. De la combinaison linéaire à la Convolution

Pour des raisons de simplicité technique et mathématique, la fonction f d'une opération locale est très souvent définie comme une **combinaison linéaire** des pixels du voisinage.

Combinaison linéaire : une somme de valeurs multipliées par des coefficients ($y = a_1x_1 + a_2x_2 + a_3x_3 + \dots$)

- **Le Noyau (Kernel)** : Lorsque l'on utilise une combinaison linéaire, la matrice qui contient les "poids" de cette combinaison est appelée un **noyau** (ou *kernel* en anglais).
- **La Convolution** : L'opération mathématique qui **applique ce noyau** sur l'image s'appelle **la convolution**. Dans l'espace des fonctions (une image étant une fonction 2D discrète), c'est **l'équivalent de l'opérateur de produit**.

Donc : combinaison linéaire + déplacement sur toute l'image = **convolution**

La formule de la convolution discrète 2D

Le résultat de la convolution d'une image I par un noyau K de taille $(2p+1) \times (2p+1)$ au pixel (i, j) est formulé ainsi :

— En 2D, discrète :
$$I * k(i, j) = \sum_{i'=-p}^p \sum_{j'=-p}^p I(i-i', j-j') k(i', j')$$
 avec k une matrice de taille $(2p+1) \times (2p+1)$

p = rayon du voisinage

on prend p pixels autour du centre dans chaque direction

Remarque cruciale du professeur : Si vous observez bien la formule, les indices de l'image sont $I(i-i', j-j')$. Les signes soustractifs signifient qu'il faut rigoureusement **prendre le symétrique du noyau** lorsqu'on effectue le produit matriciel sur l'image.

4. Algorithme de calcul : La convolution en pratique

Comment cette formule s'applique-t-elle concrètement pour calculer la nouvelle valeur d'un pixel ? L'opération repose sur une **double boucle**. Prenons l'exemple d'un noyau de dimension (3×3) (dont les indices i' pour la hauteur et j' pour la largeur vont de -1 à 1 (**donc $p = 1$**)) :

1. **Alignement** : On superpose le centre du noyau (coordonnées $(0,0)$) sur le pixel (i,j) de l'image que l'on souhaite traiter.
2. **Double Itération** : On parcourt les éléments du noyau ligne par ligne et colonne par colonne :
 - On commence par en haut à gauche : $i'=-1$ et $j'=-1$.
 - On multiplie la valeur du noyau à cet emplacement par la valeur du pixel de l'image correspondante donc le $I(i-(-1), j-(-1)) * k(-1, -1) = I(i+1, j+1) * k(-1, -1)$.
 - On passe à la colonne suivante $j'=0$, on multiplie, on additionne au résultat précédent, etc.
3. **Résultat** : Une fois la double somme terminée, la combinaison linéaire nous donne une **valeur finale**.

5. Points clés à retenir et conclusion

- **L'objectif principal** de la convolution est de remplacer chaque pixel par une combinaison de son voisinage immédiat, permettant ainsi de prendre en compte le contexte visuel.
- **L'outil central** est le **noyau (kernel)**, une matrice définissant le poids accordé à chaque voisin.
- **Perspectives** : Si utiliser des noyaux avec des chiffres purement aléatoires ou arbitraires n'a que peu d'utilité pratique, le choix de **noyaux spécifiques** (sujet de la suite du cours) permettra d'accomplir des transformations d'image puissantes (comme le lissage, la détection de contours, le renforcement des détails, etc.).

Remarque :

presque **toutes les opérations locales linéaires en image sont des convolutions** (au sens large),
mais mathématiquement strictement : il faut distinguer **convolution** et **corrélacion**(la plus utilisé pratiquement).

✓ 1. "Au sens large" (usage en image)

En traitement d'image, on appelle souvent "convolution" :

👉 toute opération locale de la forme :

- somme pondérée des pixels voisins
- mêmes poids appliqués partout dans l'image

Exemple :

- flou
- détection de contours
- filtres Sobel

📌 Même si mathématiquement ce n'est pas exactement une convolution stricte.

✓ 2. "Mathématiquement strict"

En maths, il y a une différence :

♦ Convolution (vraie définition)

$$(I * K)(i, j) = \sum K(u, v) I(i - u, j - v)$$

👉 le noyau est retourné (inversé)

♦ Corrélation (ce qu'on code souvent)

$$(I * K)(i, j) = \sum K(u, v) I(i + u, j + v)$$

👉 pas de retournement du noyau

🔴 3. Pourquoi la confusion ?

Parce que :

- si le noyau est symétrique (ex : flou gaussien, Sobel vertical approximé)
 - 👉 convolution = corrélation

Donc en pratique :

👉 on ne voit pas la différence

Cours : Traitement des images - Le Débruitage :

1. Introduction au Débruitage

Le débruitage est une opération fondamentale de **pré-traitement** (preprocessing) en traitement d'images.

- **Définition** : Il s'agit d'une opération consistant à supprimer tous les éléments indésirables ou parasites (le "bruit") présents sur une image.
- **Exemple classique** : Le "grain" photographique qui apparaît lorsque l'on prend une photo de nuit (en conditions de faible luminosité).
- **Remarque** : Les smartphones modernes produisent aujourd'hui beaucoup moins de bruit nocturne qu'auparavant, car ils intègrent nativement de puissants algorithmes de débruitage basés sur les techniques expliquées ci-dessous.

2. Le Filtre Moyenneur (Filtre Linéaire)

C'est le filtre le plus simple à concevoir pour atténuer le bruit.

- **Principe** : La nouvelle valeur d'un pixel devient la **moyenne arithmétique** de toutes les valeurs des pixels de son voisinage local.
- **Implémentation par convolution** : On utilise un **noyau (kernel)** où chaque élément possède la même pondération. Pour éviter d'augmenter artificiellement la luminosité globale de l'image, il faut **normaliser** le noyau.
 - **Exemple** : pour un noyau (3*3) (9 pixels) : On remplit la matrice avec des **1**, puis on divise le tout par la somme des éléments, soit **1/9**. Chaque case du noyau vaut donc 1/9.
- **Résultat visuel** : Le filtre moyenneur permet de lisser l'image et d'effacer les petits artefacts (comme le grain ou de fines lignes parasites).
- **Inconvénient majeur** :
 - Il introduit énormément de **flou**.
 - Plus la taille du noyau **augmente** (passant de 3*3 à 15 *15, etc.), plus l'image est **débruitée**, mais plus elle devient **illisible et floue**.

3. La gestion des bords de l'image : Le "Padding"

Le problème : Lors de l'application d'un **filtre de convolution**, lorsque le **noyau** se trouve sur un pixel au bord de l'image (ex: en haut à gauche), une partie du noyau **déborde** dans le vide. Il faut donc inventer des valeurs pour ces pixels manquants afin de calculer l'opération.

Le professeur présente 4 techniques de "padding" (remplissage) :

Exemple :

```
10 20 30
40 50 60
70 80 90
```

1. Le Zero Padding (Remplissage par des zéros) :

- **Principe** : On considère que tout ce qui dépasse de l'image vaut 0.
- **Conséquence** : Lors du calcul de la moyenne, l'intégration de ces zéros fait chuter drastiquement la valeur du pixel. Cela crée un **effet de vignettage** (apparition d'une bande noire assombrie tout autour des contours de l'image).

```
0 0 0
0 10 20
0 40 50
```

2. Le Copy Padding (Copie du pixel le plus proche) :

- **Principe** : Pour combler le vide, on copie simplement la valeur du pixel situé sur la bordure extrême de l'image.
- **Conséquence** : Supprime l'effet de vignettage indésirable du zéro padding. Idéal si les bords de l'image sont relativement homogènes.

```
10 10 20
10 10 20
40 40 50
```

3. Le Mirror Padding (Effet Miroir) :

- **Principe** : On copie les pixels de l'image en créant une symétrie (un miroir) par rapport à la bordure de l'image.
- **Conséquence** : C'est une excellente technique qui permet d'éviter les effets de bord de manière naturelle.

```
50 40 50
20 10 20
50 40 50
```

4. Le Wrapping (Padding Cyclique) :

- **Principe** : On récupère les valeurs du bord opposé de l'image (ce qui sort à gauche va chercher les valeurs de droite).
- **Cas d'usage** : À utiliser uniquement si l'image présente un motif cyclique/répétitif.

```
90 70 80
30 10 20
60 40 50
```

4. Le Filtre Gaussien

Pour pallier le problème de **flou excessif** du filtre moyenneur, on utilise le filtre gaussien.

- **Principe** : C'est un filtre de convolution dont les poids dans le noyau **ne sont pas uniformes**. Le pixel central a le coefficient le plus fort, et plus on **s'éloigne** du centre, plus la **pondération diminue** en suivant une **courbe en cloche** (une distribution gaussienne 2D).
- **Les paramètres clés** :
 - La **taille du noyau** : (ex : 7x7).
 - Le **Sigma (σ)**, qui correspond à l'**écart-type de la gaussienne** :
 - Si σ est **grand** (ex : $\sigma = 5$) : la **courbe est large**. On prend en compte un grand nombre de voisins, ce qui **augmente le flou** mais permet de **mieux réduire le bruit**.
 - Si σ est **petit** (ex : $\sigma = 1$) : la **courbe est très pointue**. Le pixel central a plus de poids, l'image **reste plus nette**, mais la **réduction du bruit est moins efficace**.
- **Avantage** : Il constitue un **excellent compromis**. Il lisse l'image et supprime les artefacts tout en préservant une meilleure netteté (**plus de piqué**) que le filtre moyenneur.

Le **piqué** = capacité d'une image à montrer :

- les contours nets
- les petits détails
- les transitions brusques (bord objets)

Remarque : Le noyau gaussien est **symétrique donc** : convolution = corrélation

- **σ (sigma)** :
 - petit → image plus nette (plus de piqué), peu de flou, bruit est moins efficace.
 - grand → moins de piqué, plus de flou, bruit mieux supprimé
- **taille du noyau** :
 - petite → peu de voisins pris en compte → image plus nette
 - grande → plus de voisins → plus de lissage (et donc plus de flou)

5. Le Filtre Médian (Filtre Non-Linéaire)

- **Principe** : On regarde toutes les valeurs des pixels dans le voisinage (fenêtre glissante), on les trie, et on remplace la valeur du pixel central par la **valeur médiane**.
- **Le Piège (Point crucial)** : Le filtre médian **n'est pas une combinaison linéaire**. Modifier drastiquement un pixel extrême (ex: rendre un pixel blanc encore plus blanc) ne changera pas la médiane. Par conséquent, **il ne peut pas être implémenté par un noyau de convolution**. Il doit être appliqué via un **algorithme de fenêtre glissante**.
- **Impact du Padding** : Étant donné qu'on ne fait que sélectionner une valeur existante (et non calculer une moyenne avec des zéros), la méthode de padding choisie est un peu moins critique, bien qu'un zéro padding créera toujours **de petits artefacts sombres dans les coins extrêmes**.
- **Avantages et Phénomènes visuels** :
 - **Préservation des contours** : Contrairement au moyenneur et au gaussien, le médian ne rend pas les contours flous. Les bords des objets restent nets.
 - **Effet "Color Block"** : Si on augmente la **taille** de la fenêtre (**ex: 19 * 19**), les **couleurs deviennent homogènes par blocs**. Les petites variations de texture (comme les détails d'un visage) sont écrasées au profit d'aplats de couleurs unies.

Synthèse et Points clés à retenir

Type de Filtre	Linéarité	Avantages	Inconvénients principaux
Moyenneur	Linéaire (Convolution)	Simple, atténue le grain	Introduit énormément de flou global .
Gaussien	Linéaire (Convolution)	Bon compromis, garde plus de "piqué"	Les paramètres (sigma et taille) doivent être bien réglés.
Médian	Non-Linéaire	Préserve parfaitement les contours nets	Effet Color block (détruit les micro-textures).

Astuce globale : Le choix de la méthode de **padding (Zero, Copy, Mirror, Wrap)** ne dépend pas d'une règle absolue, mais s'adapte en fonction de la nature de l'image d'entrée et du résultat recherché (éviter le vignettage favorise souvent le **Mirror ou le Copy padding**).

Cours : Traitement des images - Les contours :

1. Prérequis : Transformée de Fourier et Complexité de la Convolution

Avant d'aborder les contours, le professeur fait un rappel fondamental sur la transformée de Fourier et son utilité mathématique dans l'optimisation des calculs d'images.

A. Le principe de la Transformée de Fourier

- **Définition** : Toute fonction **périodique** (et par extension, la plupart des signaux si l'on prend une période infinie) peut se décomposer sous la forme d'une **somme infinie de fonctions sinusoïdales**.
- **Propriété clé** : C'est une opération **inversible**. On peut passer du domaine spatial (l'image classique) au domaine fréquentiel (Fourier), y effectuer des modifications, puis revenir au domaine spatial sans perte.

B. L'avantage en termes de complexité algorithmique

Appliquer une convolution de manière **naïve** dans le domaine spatial est extrêmement lourd.

Complexité naïve (spatiale) :

L'**algorithme** utilise une double boucle pour parcourir l'**image** de taille **m × m**, et **pour chaque pixel** une autre **double boucle** pour appliquer le **noyau** de taille **n × n**.

La complexité totale est quadratique : $\Theta(m^2 \times n^2)$.

- **Optimisation par Fourier** :

Une propriété mathématique montre que la convolution dans le domaine spatial devient une simple multiplication dans le domaine fréquentiel **F(I) * F(K)**.

- **Complexité optimisée (fréquentielle)** :

Grâce à la **Fast Fourier Transform (FFT)**, on peut passer dans le domaine de Fourier en un temps quasi linéaire $\Theta(n \log n)$.

Donc c'est $\Theta(m \log m + n \log n)$

Conclusion :

En pratique, on n'utilise quasiment jamais l'**approche spatiale naïve** avec boucles imbriquées. On privilégie la **FFT** pour réduire fortement le temps de calcul.

2. Fréquences et Filtres : Passe-bas vs Passe-haut

Les sinusoïdales issues de la transformée de Fourier représentent **différentes "fréquences" de l'image**.

- **Basses fréquences (Le signal global)** : Elles correspondent aux grandes zones de couleurs et à l'**aspect général de l'image** (comme les basses en musique).
 - **Outil** : Le **filtre passe-bas** (ex: filtre moyenneur). Il conserve le signal global mais détruit les détails (crée un effet de flou).
- **Hautes fréquences (Les détails)** : Elles correspondent aux **changements brusques dans l'image**, c'est-à-dire **les détails, le bruit... et les contours**.
 - **Outil** : Le **filtre passe-haut** (ex: le Laplacien). Il soustrait le signal global pour ne garder que les variations brusques. Le Laplacien (**3 * 3**) multiplie le **pixel central par 8** et soustrait la valeur des 8 voisins (**-1 dans les autres**). Sur une zone homogène, son résultat est nul ou presque (très faible). Sur un contour, il "s'allume" fortement.
 - noyau est **symétrique donc** : convolution = corrélation.

3. Qu'est-ce qu'un contour ?

- **Définition visuelle** : Les contours **définissent la structure des objets**. Même avec une image réduite à ses seuls contours (ex: dessin au trait), un humain peut immédiatement reconnaître le sujet (ex: un chat). C'est essentiel pour la reconnaissance de formes.
- **Définition mathématique** : Un contour est une **discontinuité** (un changement brutal d'intensité des pixels) dans un voisinage donné. On passe très vite d'une valeur faible (noir) à une valeur forte (blanc).
- **Notion d'échelle** : La définition d'un contour dépend du niveau de zoom. Une arête nette de loin peut apparaître comme un dégradé continu si l'on zoome à l'échelle du pixel. Le choix de l'outil dépendra donc du type d'image analysé.
- **arête** : une zone de transition rapide dans l'image

4. Extraction de contours par Dérivée Spatiale (Le Gradient)

La première méthode logique pour trouver un changement brusque (une pente) est de calculer une dérivée spatiale.

A. Le filtre Gradient basique

En chaque pixel, on calcule un vecteur 2D contenant deux composantes (les deux dérivées partielles), c'est la Gradient :

1. **Gradient horizontal** : Dérivée partielle par rapport à J (différence entre un pixel et son voisin de droite).
2. **Gradient vertical** : Dérivée partielle par rapport à I (différence entre un pixel et son voisin du dessous).

$$G(i, j) = \begin{pmatrix} \frac{\partial I}{\partial i} \\ \frac{\partial I}{\partial j} \end{pmatrix}$$

Gradient Horizontal			Gradient Vertical		
0	0	0	0	0	0
1	-1	0	0	-1	0
0	0	0	0	1	0

Calcul de la magnitude globale :

Pour fusionner ces deux informations en une seule image des contours, on calcule la magnitude du gradient :

Magnitude = $\sqrt{G_x^2 + G_y^2}$

— Contour = discontinuité forte, on regarde la magnitude du gradient ($|G| = \sqrt{\frac{dI^2}{di} + \frac{dI^2}{dj}}$)

- **Inconvénients du Gradient simple :**
 - Il a beaucoup de mal à détecter les contours en diagonale.
 - Il est **très sensible au bruit**, car il ne se base que sur 2 pixels voisins à la fois.

5. Les filtres avancés (Sobel et Prewitt)

Pour **corriger l'hyper-sensibilité** au bruit du gradient simple, on **élargit** la taille du noyau de convolution (prendre en compte **plus de voisins**).

- **Filtre de Sobel** : Il utilise un noyau (3 * 3). En plus de chercher la dérivée, il applique un léger lissage local.

Sobel Horizontal			Sobel Vertical		
1	0	-1	1	2	1
2	0	-2	0	0	0
1	0	-1	-1	-2	-1

- **Avantages** : Beaucoup plus robuste face au bruit, et capable de mieux détecter les lignes diagonales.
- **La méthode de magnitude** (racine carrée de la somme des carrés) reste la même.
- **Filtre de Prewitt** : C'est une **variante** très similaire à Sobel, avec des **valeurs de pondération légèrement différentes dans la matrice (faible par rapport au sobel)**.

Prewitt Horizontal			Prewitt Vertical		
1	0	-1	1	1	1
1	0	-1	0	0	0
1	0	-1	-1	-1	-1

- **La méthode de magnitude** (racine carrée de la somme des carrés) reste la même.
- Le choix entre **Sobel**, **Prewitt** ou le **Laplacien** dépend de l'application visée (ex: détecter les micro-détails de brins d'herbe (sensible aux bruits Laplacien et peut être Gradient (pas mal)) ou seulement les murs des bâtiments (sobel)).

6. L'algorithme complet : Le filtre de Canny

C'est la méthode de référence, la plus aboutie et toujours massivement utilisée aujourd'hui, car elle combine intelligemment plusieurs étapes pour obtenir des contours parfaits (binaires et nets).

Les 4 étapes de l'algorithme de Canny :

1. **Filtre Gaussien (Passe-bas)** : C'est paradoxal, mais on commence par "flouter" légèrement l'image. *Objectif : éliminer le bruit et les micro-détails inutiles qui fausseraient la détection.*
2. **Calcul de la magnitude du Gradient** : On applique un filtre (comme Sobel) pour trouver les zones de fortes variations.
3. **Suppression des non-maxima locaux (Non-maximum suppression)** : L'algorithme affine les bords détectés à l'étape 2 (qui sont souvent trop épais). Il ne garde que le "pic" d'intensité pour affiner le contour à la largeur d'un seul pixel.
4. **Seuillage (Thresholding)** : On applique un seuil final sur la magnitude pour transformer le résultat en une image purement binaire (noir ou blanc). Le contour est formellement validé.

Conclusion et Applications Modernes

- Bien que classique, la détection de contours (notamment avec Canny) reste un pilier de la vision par ordinateur.
- **Remarque d'ouverture** : Le professeur précise que ces techniques sont même utilisées dans l'Intelligence Artificielle générative moderne (Modèles de Diffusion). On peut extraire les contours de Canny d'une image pour forcer l'IA à générer une nouvelle scène (avec des textures différentes) tout en respectant strictement la géométrie et l'agencement spatial d'origine.

Cours : Traitement des images – Morphologie mathématique :

1. Introduction et Contexte

La **morphologie mathématique** est une théorie fondamentalement basée sur la théorie des ensembles. Elle est très souvent intégrée dans les chaînes de traitement d'images.

Historique : Cette théorie a été inventée dans les années 1960 par Georges Matheron et Jean Serra à l'École des Mines de Paris.

Domaines d'application : Conçue à l'origine pour l'étude et l'analyse des roches, elle est aujourd'hui massivement utilisée dans divers domaines tels que la biologie, l'imagerie médicale, la télédétection et les sciences des matériaux.

2. Bases Mathématiques : Opérations sur les ensembles

Pour comprendre les opérations morphologiques, il faut d'abord maîtriser les opérations ensemblistes de base appliquées à des images (notamment binaires) :

L'Union ($\cup$)

L'union de deux ensembles I et S regroupe tous les éléments appartenant à I , à S , ou aux deux.

Formule : $I \cup S = \{x \mid x \text{ appartient à } I \text{ ou } x \text{ appartient à } S\}$

Exemple : Si $A = \{(0,0), (1,1), (0,1)\}$ et $B = \{(0,1), (1,0)\}$, alors $A \cup B = \{(0,0), (1,1), (0,1), (1,0)\}$.

L'Intersection ($\cap$)

L'intersection ne conserve que les éléments qui sont strictement communs aux deux ensembles.

Formule : $I \cap S = \{x \mid x \text{ appartient à } I \text{ et } x \text{ appartient à } S\}$

Exemple : Avec les mêmes ensembles A et B , l'intersection donne $A \cap B = \{(0,1)\}$

3. Les Opérations de Minkowski

Ces opérations sont le socle mathématique qui permet de définir la dilatation et l'érosion.

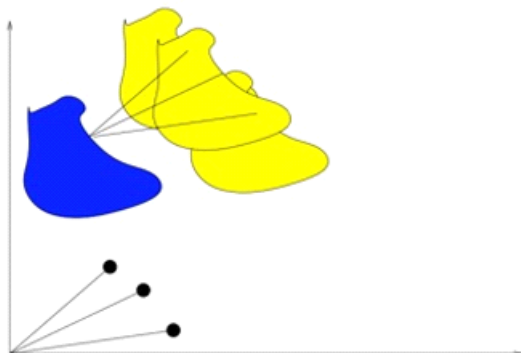
La Somme de Minkowski ($\oplus$)

Elle consiste à additionner **chaque point** du premier ensemble avec **chaque point** du second ensemble.

Formule : $I \oplus S = \{i + s \mid i \text{ appartient à } I, s \text{ appartient à } S\}$

Alternative : Elle peut aussi s'écrire comme l'union de l'ensemble X translaté par chaque vecteur b de l'ensemble B .

Formule : $X \oplus B = \bigcup_{b \in B} (X + b)$

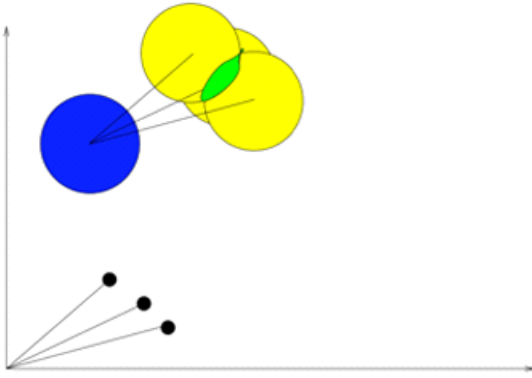


La Soustraction de Minkowski ($\ominus$)

C'est l'opération inverse, consistant à chercher l'intersection de toutes les translations.

Formule : $I \ominus S = \{x \mid x + s \in I \text{ pour tout } s \in S\}$

Formule : $X \ominus B = \bigcap_{b \in B} (X + b)$



4. L'Outil Central : L'Élément Structurant

Dans le cas des images numériques, les opérations de base se font à l'aide d'un **outil** géométrique que l'on déplace sur l'image.

Définition : Un élément structurant (**noté S**) est un sous-ensemble de l'espace de l'image (ici $\mathbb{Z}^2$) (**cad les coordonnées**) centré sur une **origine de coordonnées (0, 0)**. Dans les schémas, cette origine est généralement représentée par une **flèche**.

- S ne **décrit pas des pixels fixes** (donc il peut avoir des coordonnées qui n'existent pas dans l'image)
- S décrit une **forme autour du pixel courant**

Application : L'élément structurant **S** appliqué/translaté à un point **x** de l'image est noté : $S_x = \{x + s \mid s \text{ appartient à } S\}$, c'est comme on décale le point x selon chaque vecteur de S.

Le Symétrique : Le symétrique d'un élément structurant, noté $S^\sim$, consiste à inverser les signes de ses coordonnées :

$S^\sim = \{-s \mid s \text{ appartient à } S\}$. C'est comme si l'on faisait **pivoter la forme de 180 degrés** autour de son origine.

5. La Dilatation

La dilatation est la première opération morphologique de base étudiée. Elle tend à **épaissir** ou **agrandir** les objets dans l'image.

A. Définitions mathématiques

Définition primaire : La dilatation d'une image **I** par un élément structurant **S** est exactement la somme de Minkowski de I et S.

$$\delta S(I) = I \oplus S = \{i + s \mid i \text{ appartient à } I, s \text{ appartient à } S\}$$

Définition par le symétrique (pratique) : On garde un pixel **i** si l'élément structurant symétrique $S^\sim$ centré en **i** touche (a une intersection non vide avec) l'image **I**.

(au moins un pixel du voisinage est égal à 1)

$$\delta S(I) = \{i \mid I \cap S^\sim i \neq \emptyset\}$$

Directe : (avec le **S** ou $S^\sim$)

On met le x ??? : Pour le **S** si le x est 1 tous les autres deviennent 1

On met le x sur les 0 : Pour le $S^\sim$ si au moins 1 des voisins de x est 1 dans I, x devient 1.

B. Effets visuels et Propriétés

Conséquences visuelles : La dilatation permet de boucher les trous dans une forme, de fusionner les composantes connexes qui sont proches l'une de l'autre, et d'augmenter globalement la taille des formes de l'image.

Propriétés mathématiques :

Croissance : Si une forme **X** est incluse dans **Y**, alors la dilatation de **X** sera incluse dans la dilatation de **Y**

$$(Si X \subset Y, \delta S(X) \subset \delta S(Y)).$$

Impact de la taille de l'outil : Si on utilise un élément structurant **S** plus grand ($S \subset S'$), le résultat de la dilatation sera plus grand ($\delta S(I) \subset \delta S'(I)$).

Composition : Dilater successivement par **S** puis par S' équivaut à dilater l'image en une seule fois par la somme de Minkowski de **S** et S' ($\delta S \oplus S'(I) = \delta S'(\delta S(I))$).

6. L'Érosion

L'érosion est l'opération opposée à la dilatation. Elle tend à **ronger** ou **rétrécir** les objets dans l'image.

A. Définitions mathématiques

Définition primaire : L'érosion d'une image I par l'élément structurant S correspond à la soustraction de Minkowski de l'image I avec le symétrique de S ($S^\sim$).

$$\varepsilon S(I) = I \ominus S^\sim = \{i \mid \text{pour tout } s \text{ dans } S, i + s \text{ appartient à } I\}$$

Définition par l'inclusion (pratique) : On conserve un pixel i uniquement si l'élément structurant S tout entier, une fois centré en i , est strictement inclus dans la forme de l'image I .

$$\varepsilon S(I) = \{i \mid S_i \subset I\}$$

Directe : (avec le S)

On met le x sur les 1 où : si tous les voisins sont 1, il reste 1 sinon il devient 0. (on ignore les 0)

B. Effets visuels et Propriétés

Conséquences visuelles : Contrairement à la dilatation, l'érosion supprime les petites composantes connexes (le bruit isolé), efface les détails situés sur les bords des formes, et augmente la taille des trous à l'intérieur des objets.

Propriétés mathématiques :

Croissance : Comme pour la dilatation, si $X \subset Y$, alors $\varepsilon S(X) \subset \varepsilon S(Y)$.

Impact de la taille de l'outil : (Selon les notes du cours) Si $S \subset S'$, alors $\varepsilon S(I) \subset \varepsilon S'(I)$. (Note : de manière empirique, un outil plus grand ronge davantage l'image).

Composition : Éroder successivement par S puis par S' équivaut à éroder directement par l'outil résultant de $S \oplus S'$ ($\varepsilon S \oplus S'(I) = \varepsilon S'(\varepsilon S(I))$).

Synthèse et Points clés à retenir

La théorie ensembliste est le cœur du fonctionnement de la morphologie mathématique.

L'Élément Structurant (S) agit comme le **pinceau** qui modifie l'image. Son origine (centre) et sa forme dictent le résultat.

Dilatation = Intersection de l'outil symétrique avec l'image (le voisinage touche au moins un pixel de l'image) ->

(Élargit et connecte).

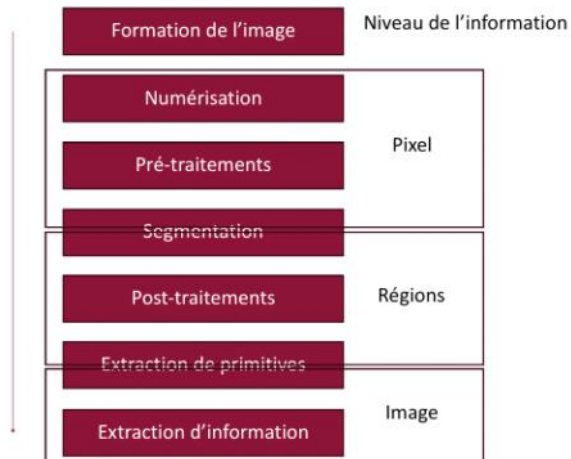
Érosion = Inclusion stricte de l'outil dans l'image (tout le voisinage est entièrement dans l'objet) -> (Rétrécit et nettoie).

Ces deux opérations constituent les briques de base absolues de toute chaîne de traitement morphologique (qui permettront par la suite de créer l'Ouverture et la Fermeture).

Traitement des Images Numériques : Méthodes Globales et Réseaux de Neurones

1. La Chaîne de Traitement des Images

Avant d'**extraire** de l'**information**, une **image** passe par plusieurs étapes standards :



- **Formation de l'image** : l'image est créée ou capturée par un capteur.
- **Numérisation** : transformer le signal capté en une image numérique composée de pixels (Echantillonnage et Quantification)
- **Pré-traitements** : améliore la qualité de l'image avant l'analyse (Suppression du bruit avec Filtrage (filtre moyenneur..))
- **Segmentation** : découper l'image en régions ou objets cohérents (Séparer (Seuillage), Regroupement (Clustering))
- **Post-traitements** : Après la segmentation, certaines erreurs existent encore, nettoyer et améliorer le résultat (Opérations morphologiques (Dilatation...))
- **Extraction de primitives** : Une primitive est une caractéristique simple détectée dans l'image, (extrait les éléments importants (Contour, Formes...))
- **Extraction d'information** : interpréter les primitives pour comprendre l'image.

Extraction d'information : C'est ici que l'intelligence artificielle ou les algorithmes prennent une décision (Classification, Régression)

2. Extraction d'Information : Méthodes Classiques vs Apprentissage

Une fois l'image traitée, on cherche à extraire l'information selon la tâche souhaitée.

Les 3 grandes tâches et leurs algorithmes classiques

Classification (catégoriser l'image = Dire ce que contient globalement l'image)

- Support Vector Machines (SVM)
- Random Forest
- K plus proches voisins (k-NN)

Segmentation (découper l'image = Séparer précisément les différentes parties de l'image)

- Clustering (ex. K-moyennes)
- Champs de Markov
- Croissance de région

Régression (prédire une valeur continue)

- Régression linéaire
- Random Forest
- K plus proches voisins (k-NN)

Le problème des "Caractéristiques" (Features)

Historiquement, parce que les algorithmes ne savaient pas "regarder" une image directement, ils reposent sur des caractéristiques choisies manuellement.

Niveau bas

- Valeur du pixel
- Contours (réponse de convolution)

Niveau haut

- Descripteurs locaux (HOG, LBP)
- Histogrammes
- Formes (critères morphologiques)

Exemple (Random Forest)

Pour distinguer un livre d'un magazine, on crée manuellement un arbre de décision basé sur des règles fixées :

- Est-ce qu'il y a moins de 15 lignes ?
- Y a-t-il une image ?
- Commence-t-il par une majuscule ?

La solution des Réseaux de Neurones

Au lieu de deviner ou **choisir manuellement** les caractéristiques pertinentes **selon l'application ou l'image**, le réseau de neurones va les **apprendre** automatiquement.

3. Le Fondement : Le Perceptron

Le **perceptron** est l'unité de base (le réseau de neurones le plus simple).

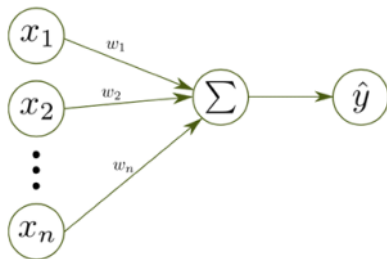
But

Apprendre une fonction **f** pour obtenir un résultat **y** (ex : prédire "ville" ou "non ville") à partir d'une entrée **x** (ex : les pixels de l'image), en essayant d'approximer une fonction idéale **f***

f* : C'est la relation mathématique **parfaite** (mais généralement inconnue) qui associe une entrée **x** à la sortie exacte **y** que l'on cherche à obtenir.

1. La Combinaison Linéaire

Le perceptron



$$\hat{y} = f(\mathbf{x}) = \sum_{i=1}^n w_i \times x_i$$

Le **neurone** effectue une **somme des entrées xi** multipliées par **des poids d'importance wi** :

$\hat{y} = \sum (w_i \times x_i)$ pour i allant de 1 à n

Autrement dit :

$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n$

2. Le Biais

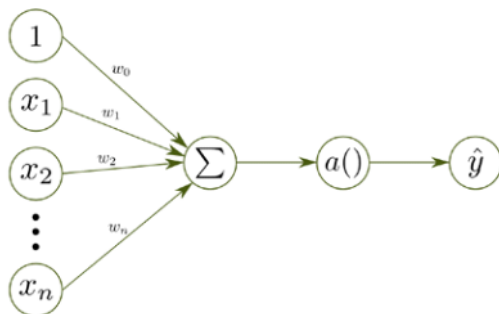
On ajoute une **constante** au calcul (un poids w_0 multiplié par 1) **pour décaler la fonction** :

$\hat{y} = \sum (w_i \times x_i)$ pour i allant de 0 à n

Autrement dit :

$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$

3. La Fonction d'Activation (a)



$$\hat{y} = f(\mathbf{x}) = a\left(\sum_{i=0}^n w_i \times x_i\right)$$

Une simple **combinaison linéaire** (qui peut tracer que des droites) est insuffisante car les **problèmes réels** sont fortement **non linéaires**.

On applique donc une **fonction non linéaire a** (comme TanH, Sigmoid/Logistique ou ReLU) au résultat de la somme afin de permettre au réseau d'approximer des fonctions complexes.

Chaque **neurone** d'arrivée possède exactement **un biais** qui lui est propre (représenté par le 1 dans ta formule).

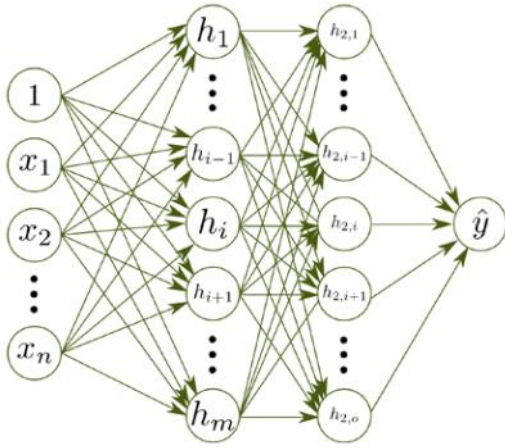
Formule finale du Perceptron

$\hat{y} = a(\sum (w_i \times x_i))$

ou :

$\hat{y} = a(w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n)$

4. Du Perceptron au Deep Learning (Réseaux Multicouches)



Pour modéliser des problèmes très complexes, on ajoute des couches cachées **hi** entre les **entrées** et la **sortie**.

Cela revient à **calculer** une **fonction non linéaire** d'une **autre fonction non linéaire** parce que :

-> La **deuxième couche** calcule donc une "**fonction non linéaire**" à partir du résultat d'une autre "**fonction non linéaire**".

On peut ajouter autant de **couches** que nécessaire, ce qui **augmente** énormément le nombre de **paramètres** à apprendre.

Remarque :

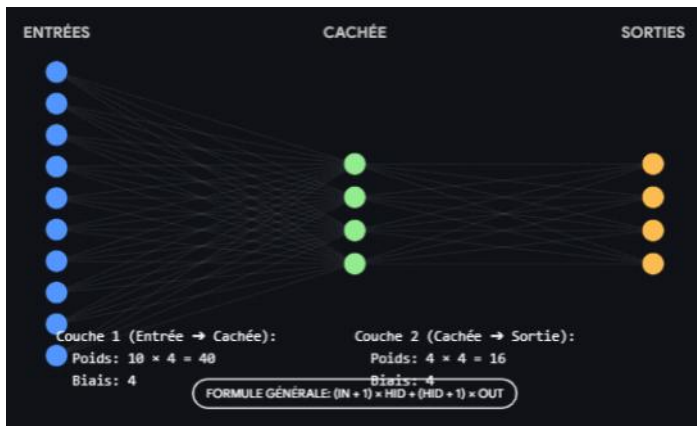
les fonctions d'activation sont cachées à l'intérieur de chaque cercle.

Dans ce schéma représentant un réseau à plusieurs **couches**, pour garder le dessin **lisible**, on ne dessine plus le détail " $\Sigma \rightarrow a()$ "

Exemple

Le modèle linguistique **GPT-3.5** possède environ **175 milliards de paramètres**.

Calculs de nombre de paramètres :



La formule magique pour relier une couche **A** à une couche **B** est donc :

Nombre de paramètres = ((Neurones de la couche précédente **A**) + 1) * Neurones de la couche actuelle **B**

5. L'Apprentissage : Comment la machine trouve-t-elle les bons poids ?

Deux options : Les fixer à la main ou Les apprendre.

Comme il y a trop de paramètres pour les régler manuellement, le réseau doit les **apprendre** à partir d'une grande quantité de **données**.

Les 3 étapes de l'apprentissage

1. Passe avant (Forward Pass)

Le réseau fait une prédiction et mesure sa qualité.

2. Erreur

Le réseau cherche d'où viennent les erreurs de prédiction.

3. Passe arrière (Backward Pass)

Le réseau met à jour ses paramètres afin de réduire l'erreur.

Estimer la qualité : La Fonction de Perte (Loss Function)

Le but est d'approximer la vraie valeur y_i avec notre prédiction $f(x_i) = \hat{y}_i$.

Exemple : MSE (Mean Squared Error)

Pour un problème de régression (par exemple deviner le nombre de lignes d'un document), on utilise souvent l'**erreur quadratique moyenne** :

$$l(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2 \quad (l = \text{Loss})$$

Risque empirique J

C'est la moyenne de toutes les erreurs sur les données d'entraînement.

L'objectif est de minimiser $J(f)$, (souvent = MSE)

—Pour un exemple: $l(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$

—Risque empirique: $J = \frac{1}{t} \sum_{i=1}^t l(y_i, \hat{y}_i)$

—Risque empirique: $J(f) = \frac{1}{t} \sum_{i=1}^t l(y_i, f(x_i))$

- t = nombre total d'exemples (taille du dataset)
- $i = 1 \rightarrow t$ = on parcourt tous les exemples
- $l(y_i, \hat{y}_i)$ = erreur sur un exemple

—En d'autres termes: minimiser $J(f) \Rightarrow$ améliorer f

—Une tâche **d'optimisation**

—Plein d'autres loss possibles, selon la tâche.

Minimiser l'erreur : La Descente de Gradient

La descente de gradient est l'algorithme utilisé pour trouver le minimum de la fonction d'erreur (fonction de coût J)

Étapes

1. On choisit des poids initiaux aléatoires.
2. On calcule le gradient (la pente de la fonction d'erreur).
3. On effectue un pas dans la direction opposée au gradient.
4. On répète ce processus jusqu'à convergence.

Apprentissage – Descente de gradient

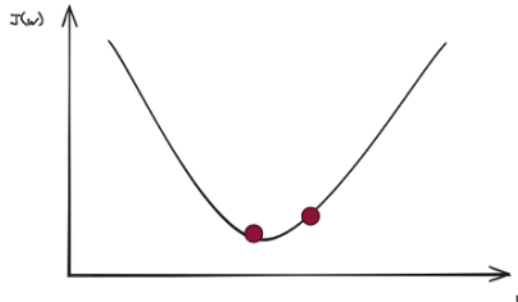
—Cas simple, un seul paramètre (w) à fixer

—Étape 1: on part d'un point initial

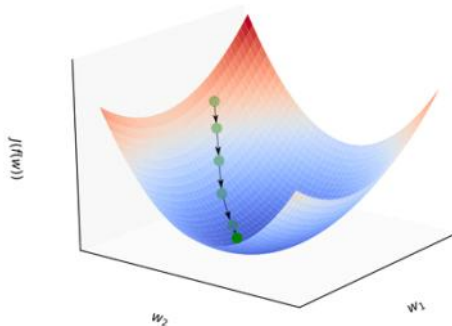
—Étape 2: calcul du gradient de ce point

—Étape 3: faire un pas dans la direction opposée au gradient

—Et on itère



Apprentissage – Descente de gradient



Note

La fonction d'erreur réelle est généralement très complexe et non convexe.

Comment calculer le gradient ? (Chain Rule)

Apprentissage – Calcul du gradient

—On souhaite calculer le gradient du risque empirique par rapport aux poids du modèle:

$$\frac{dJ(f)}{df} = \frac{dJ(f(\mathbf{w}))}{d\mathbf{w}}$$

—Cas illustré:

$$\hat{y} = f(w)(x_1) = w * x_1$$

$$J(f(w)) = (y_1 - (w * x_1))^2$$

$$\frac{dJ(f(w))}{dw} = -2x_1(y_1 - w * x_1)$$



Apprentissage – Calcul du gradient

—On souhaite calculer le gradient du risque empirique par rapport aux poids du modèle

—Cas illustré:

$$\frac{dJ(f(w))}{dw} = -2x_1(y_1 - w * x_1)$$

—Mise à jour poids:

$$l_r * (-\frac{dJ(f(w))}{dw}(x_1))$$

—Exemple avec $l_r = 0.01$

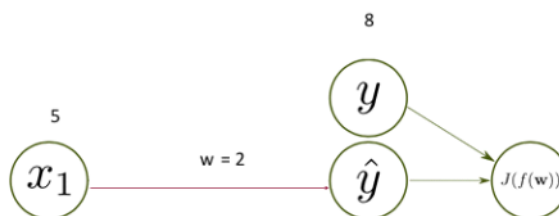
$$\hat{y} =$$

$$J(f(w)) =$$

$$\frac{dJ(f(w))}{dw}(x_1)$$

$$w =$$

$$\hat{y} =$$



Remplissage des lignes :

- $\hat{y} = 10$
(Calcul : la prédiction initiale est $\hat{y} = w \times x_1 = 2 \times 5 = 10$)
- $J(f(w)) = 4$
(Calcul : on calcule l'erreur avec la formule $J = (y_1 - \hat{y})^2 = (8 - 10)^2 = (-2)^2 = 4$)
- $\frac{dJ(f(w))}{dw} = 20$
(Calcul du gradient : on applique la formule donnée sur la diapo $-2x_1(y_1 - w \times x_1)$ ce qui donne $-2 \times 5 \times (8 - 10) = -10 \times -2 = 20$)
- $w = 1.8$
(Calcul de la mise à jour du poids : On prend l'ancien poids et on lui soustrait le gradient multiplié par le taux d'apprentissage. $w_{\text{nouveau}} = w_{\text{ancien}} - (l_r \times \text{gradient}) = 2 - (0.01 \times 20) = 2 - 0.2 = 1.8$)
- $\hat{y} = 9$
(Calcul : on fait une nouvelle prédiction avec notre tout nouveau poids. $\hat{y}_{\text{nouveau}} = 1.8 \times 5 = 9$)

Conclusion de l'exercice :

Au début, la prédiction du réseau était de 10 (alors que la cible était 8).

Après une seule étape d'apprentissage (calcul du gradient et mise à jour du poids), la nouvelle prédiction est de 9. Le modèle a corrigé son erreur et s'est rapproché de la bonne réponse !

Pour savoir comment modifier chaque poids individuellement, on utilise la règle des dérivées composées (Chain Rule). On remonte depuis le résultat final jusqu'au poids initial.

Exemple

$$\partial J / \partial w_1 = (\partial J / \partial \hat{y}) \times (\partial \hat{y} / \partial h_1) \times (\partial h_1 / \partial w_1)$$

Apprentissage – Calcul du gradient

—On souhaite calculer le gradient du risque empirique par rapport aux poids du modèle:

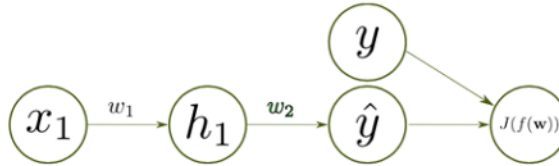
$$\frac{dJ(f)}{df} = \frac{dJ(f(\mathbf{w}))}{d\mathbf{w}}$$

—Il faut calculer la contribution de chaque poids.

—Comment ?

—Fonction composée! (a.k.a. chain rule)

$$\frac{\partial J(f(\mathbf{w}))}{\partial w_1} = \frac{\partial J(f(\mathbf{w}))}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_1} = \frac{\partial J(f(\mathbf{w}))}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h_1} \times \frac{\partial h_1}{\partial w_1}$$



La Règle Générale (en français simple)

Pour connaître l'impact d'un paramètre spécifique sur l'erreur finale, il faut multiplier l'impact (la dérivée) de **toutes les étapes intermédiaires** qui relient ce paramètre à l'erreur finale, en remontant à l'envers.

Peu importe qu'il y ait 1 couche cachée ou 100 couches cachées. Si le réseau est très profond, la chaîne de multiplications sera juste plus longue !

La Formule Générale (Mathématiques)

Si on imagine un poids w qui se trouve dans une certaine couche (disons la couche h_k), et que l'information doit traverser plusieurs autres couches (h_{k+1} , h_{k+2} , etc.) avant d'arriver à la prédiction $\hat{y}$ puis à l'erreur J , la formule est une grande multiplication en cascade :

$$\frac{\partial J}{\partial w} = \underbrace{\frac{\partial J}{\partial \hat{y}}}_{\text{Erreur finale}} \times \underbrace{\frac{\partial \hat{y}}{\partial h_{\text{dernière}}} \times \dots \times \frac{\partial h_{k+1}}{\partial h_k}}_{\text{Toutes les couches traversées}} \times \underbrace{\frac{\partial h_k}{\partial w}}_{\text{Le poids visé}}$$

6. Application aux Images : Les CNN (Réseaux de Neurones Convolutifs)

Le problème avec les réseaux classiques

Pour donner une image à un réseau classique, il faut transformer l'image en un grand vecteur plat de pixels.

Défauts

- Le contexte spatial est perdu.
- Les relations entre pixels voisins disparaissent.
- Le nombre de paramètres devient énorme.

Exemple

Une image de 1000×1000 pixels traitée par deux couches de 1000 neurones génère :

$10^6 \times 10^3 = 10^9$ paramètres

soit plus d'un milliard de paramètres.

La Solution : Les CNN

Les réseaux convolutifs reposent sur deux idées principales :

1. Le contexte spatial dans une image est important.
2. Les motifs visuels se répètent souvent.

L'Opération de Convolution

Au lieu d'analyser toute l'image d'un coup, on applique de petits filtres sur des zones locales de l'image.

Ces filtres réalisent :

- des combinaisons linéaires,
- suivies de fonctions d'activation.

Le résultat produit des « feature maps » (cartes de caractéristiques).

L'Opération de Pooling

Le pooling est une opération mathématique fixe (non apprise par le réseau), comme :

- Average Pooling (moyenne),
- Max Pooling (maximum).

Son rôle est de réduire la taille des données afin :

- d'augmenter le champ réceptif,
- de rendre le réseau plus robuste,
- de diminuer le nombre de calculs.

Architecture Typique d'un CNN

Un CNN alterne généralement entre :

- des couches de convolution,
- des couches de pooling.

À la fin :

1. les données sont aplaties,
2. puis envoyées dans un perceptron classique,
3. afin de produire la sortie finale.

Comment le réseau “voit” les images ? (Représentation hiérarchique)

Les différentes couches apprennent des niveaux d'information différents.

Premières couches

Elles détectent des éléments simples :

- lignes,
- contours,
- couleurs.

Couches intermédiaires

Elles combinent ces éléments pour détecter :

- textures,
- parties d'objets,
 - roues,
 - fenêtres,
 - portes, etc.

Dernières couches

Elles reconnaissent l'objet complet.

Exemple

Le concept global d'une « Audi A7 ».

Conclusion sur les CNN

Les CNN sont très puissants, mais ils ne sont pas « magiques ».

Ils reposent sur des concepts mathématiques classiques :

- convolutions,
- activations,

adaptés aux propriétés des images :

- contexte spatial,
- répétition des motifs.

Évolution actuelle

Aujourd'hui, les CNN commencent progressivement à être concurrencés ou remplacés par les architectures de type Transformers.